Chương 8

QUẢN LÝ BỘ NHỚ

NỘI DUNG

1. Kiến thức cơ sở
2. Cấp phát bộ nhớ liên tục
3. Phân trang
4. Cấu trúc bảng phân trang
5. Hoán đổi tiến trình
6. Bộ nhớ ảo

MỤC TIÊU

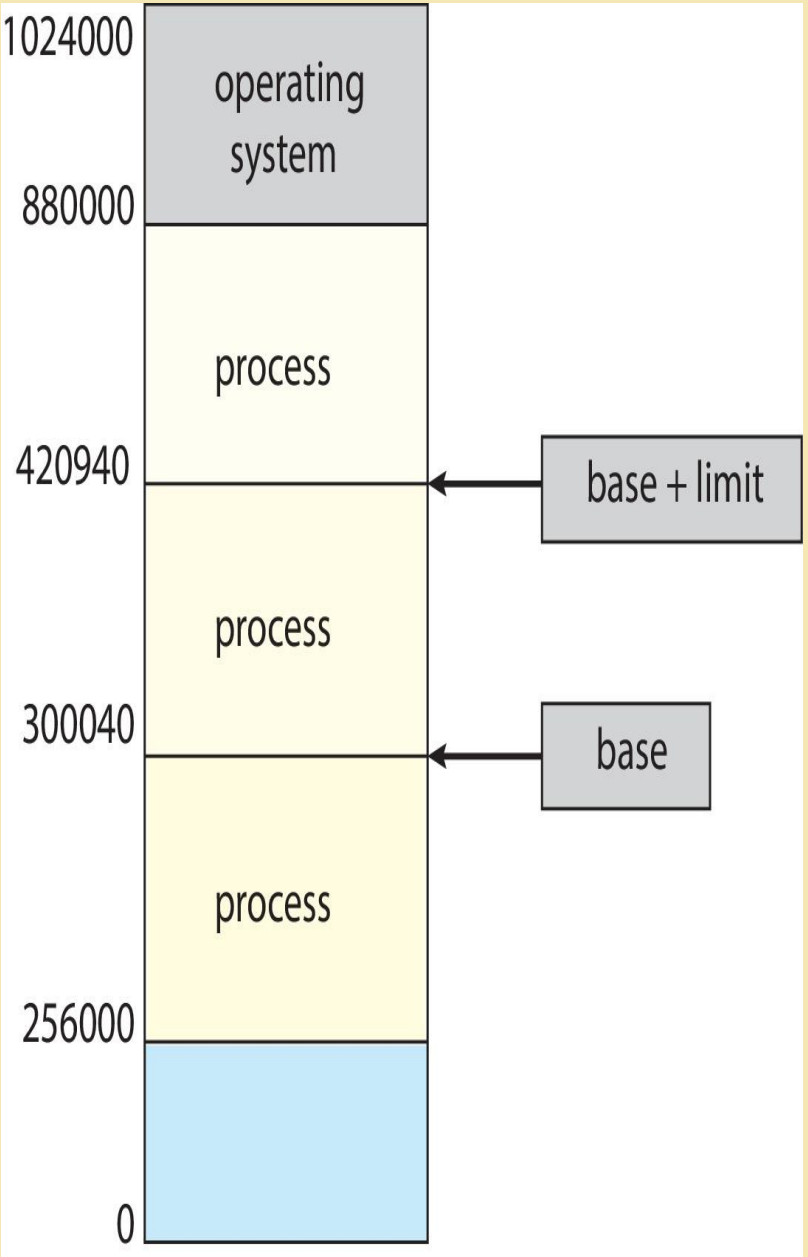
- ✓ Mô tả chi tiết các cách tổ chức khác nhau của bộ nhớ vật lý
- ✓ Thảo luận các phương pháp quản lý bộ nhớ khác nhau
- ✓ Mô tả bộ nhớ ảo và lợi ích của nó.
- ✓ Áp dụng các thuật toán thay thế trang, FIFO, tối ưu và LRU trong quản lý bộ nhớ ảo

1. Kiến thức cơ sở

- Chương trình phải được mang từ đĩa vào bộ nhớ và được tổ chức theo tiến trình để chạy
- Bộ nhớ chính và các thanh ghi là bộ lưu trữ duy nhất mà CPU có thể truy cập trực tiếp
- Đơn vị nhớ chỉ thấy dòng địa chỉ trong bộ nhớ
- Truy cập thanh ghi chỉ trong suốt 1 chu kỳ đồng hồ của CPU (hoặc ít hơn)
- Truy cập bộ nhớ chính có thể tốn nhiều chu kỳ đồng hồ của CPU
- **Cache** được đặt giữa bộ nhớ chính và các thanh ghi CPU
- Việc bảo vệ bộ nhớ được yêu cầu để đảm bảo các thao tác chính xác

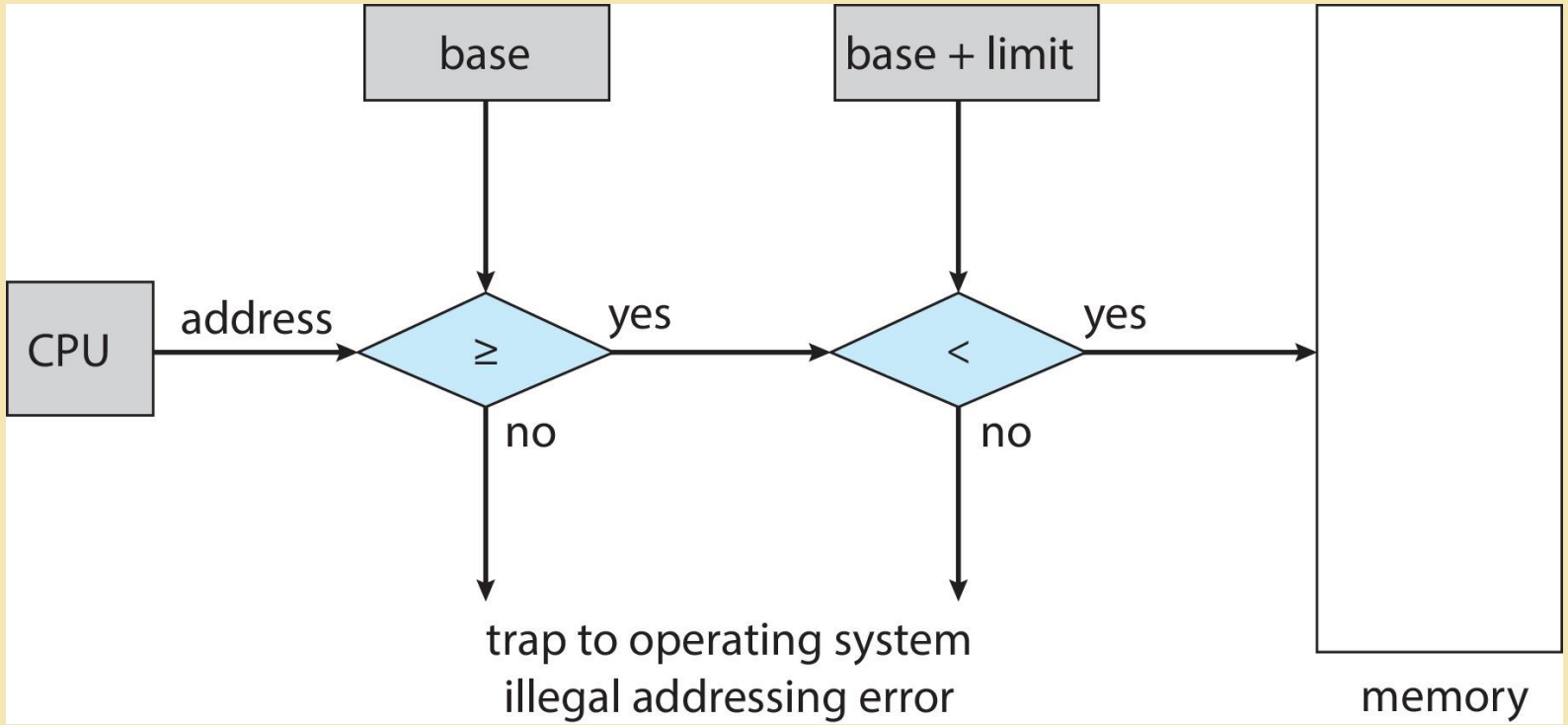
Bảo vệ địa chỉ bộ nhớ

- Mỗi tiến trình chỉ được phép truy cập không gian địa chỉ hợp pháp của nó.
- Cặp thanh ghi nền **(base register)** và thanh ghi giới hạn **(limit register)** xác định không gian địa chỉ hợp pháp cho tiến trình.
  - Thanh ghi cơ sở giữ địa chỉ bộ nhớ vật lý hợp pháp nhỏ nhất
  - Thanh ghi giới hạn chỉ định kích thước của vùng nhớ



Bảo vệ địa chỉ bộ nhớ

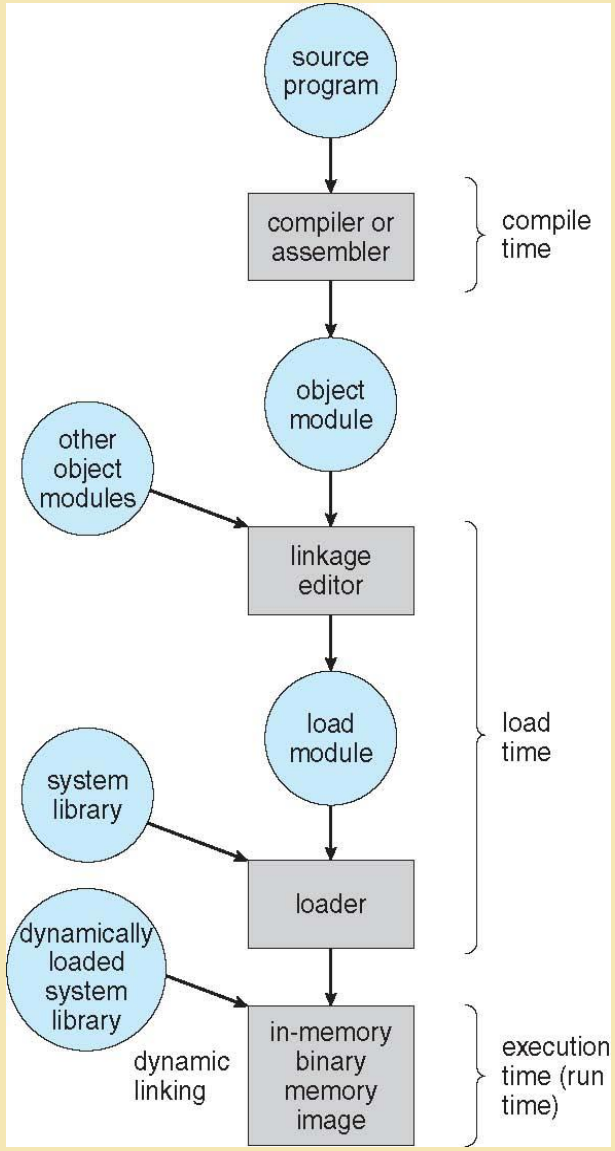
- CPU phải kiểm tra hết các truy cập bộ nhớ tạo ra trong chế độ người dùng để chắc chắn nó ở giữa thanh ghi nền và thanh ghi giới hạn.
- Các thanh ghi nền và thanh ghi giới hạn chỉ có thể được tải bởi hệ điều hành, sử dụng một lệnh đặc quyền.



Kết buộc địa chỉ (Address Binding)

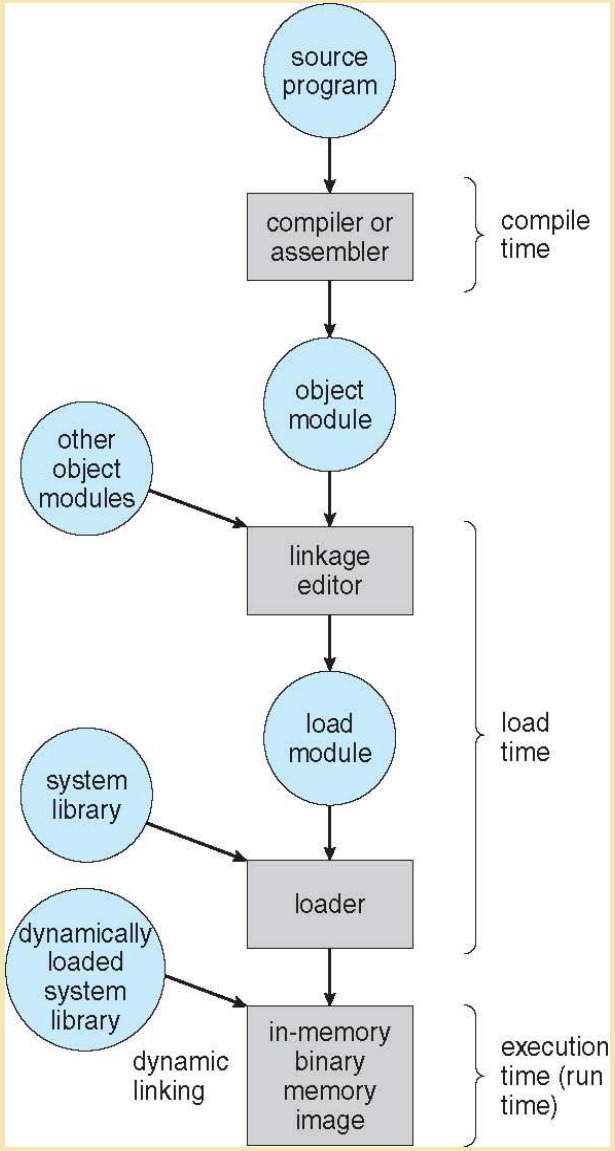
- Hàng đợi nhập (input queue): Các chương trình trên đĩa đang chờ được đưa vào bộ nhớ để thực thi
- Một chương trình trải qua nhiều bước trước khi được thực thi
- Địa chỉ được thể hiện theo những cách khác nhau ở các giai đoạn khác nhau trong suốt quá trình chạy của chương trình
  - Địa chỉ chương trình nguồn là địa chỉ tượng trưng (symbolic addresses)
  - Địa chỉ tượng trưng được trình biên dịch kết buộc tới địa chỉ tương đối (relocatable addresses)
  - Trình liên kết (linker) hoặc trình nạp (loader) sẽ kết buộc địa chỉ tương đối với địa chỉ tuyệt đối (absolute addresses)

Kết buộc địa chỉ là ánh xạ một không gian địa chỉ tới một không gian địa chỉ khác



Kết buộc chỉ thị và dữ liệu vào bộ nhớ

- Kết buộc địa chỉ của chỉ thị và dữ liệu vào địa chỉ bộ nhớ có thể xảy ra ở 3 thời điểm khác nhau
  - Thời điểm biên dịch: Nếu vị trí bộ nhớ được biết trước, mã tuyệt đối (absolute code) được tạo ra; Phải biên dịch lại mã nếu vị trí bắt đầu thay đổi.
  - Thời điểm nạp: Phải tạo mã tương đối nếu vị trí bộ nhớ không được biết ở thời điểm biên dịch.
  - Thời điểm xử lý: Nếu tiến trình được di chuyển từ một phân đoạn bộ nhớ sang một phân đoạn khác trong suốt quá trình thực thi của nó, thì việc kết buộc được trì hoãn đến thời điểm xử lý.
    - Cần phần cứng hỗ trợ cho việc ánh xạ địa chỉ (vd: thanh ghi nền và thanh ghi giới hạn)

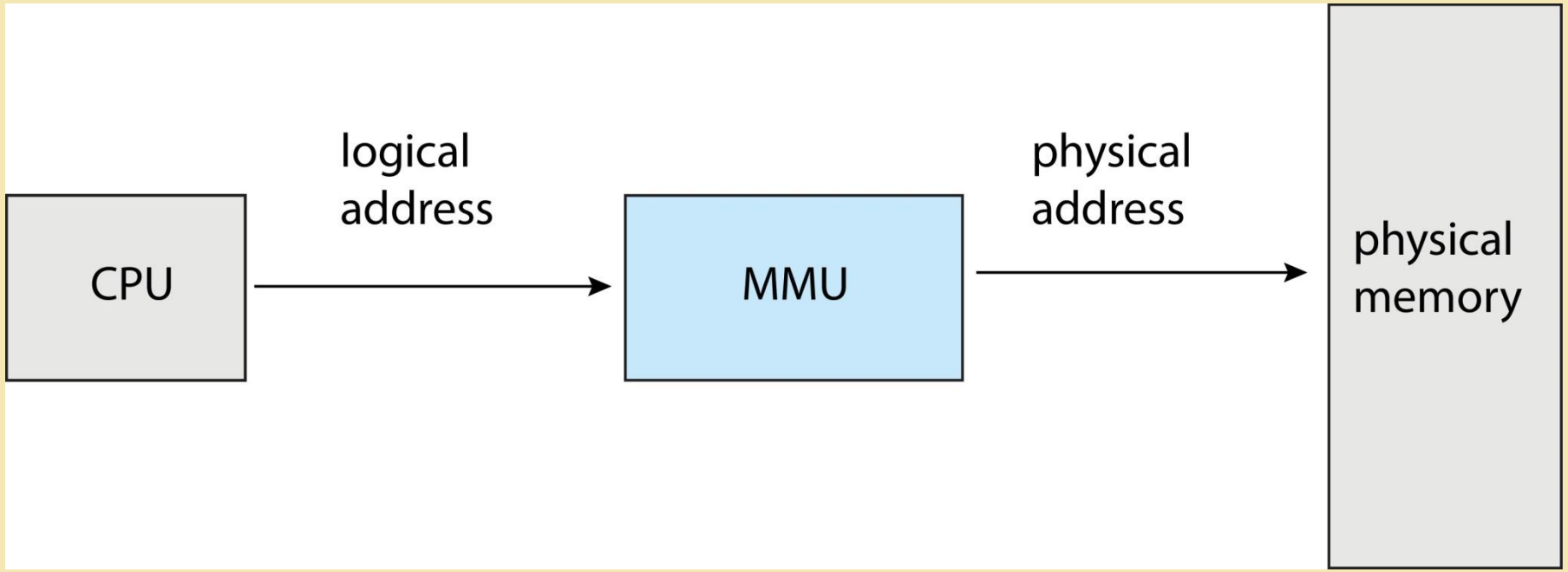


Không gian địa chỉ logic & không gian địa chỉ vật lý

- Việc tách rời khái niệm không gian địa chỉ logic (Logical address space) được xây dựng trên không gian địa chỉ vật lý (Physical address space) là trung tâm của việc tổ chức quản lý bộ nhớ một cách hiệu quả.
  - Địa chỉ logic (Logical address) – được tạo ra bởi CPU, cũng được gọi là địa chỉ ảo (virtual address)
  - Địa chỉ vật lý (physical address) – địa chỉ được nhìn thấy bởi đơn vị bộ nhớ
- Địa chỉ ảo và địa chỉ vật lý là như nhau trong phương thức kết buộc địa chỉ vào **thời điểm biên dịch** cũng như vào **thời điểm nạp**. Nhưng khác nhau vào **thời điểm xử lý**.
- Không gian địa chỉ logic – tập hợp tất cả các địa chỉ logic được tạo ra bởi một chương trình
- Không gian địa chỉ vật lý – tập hợp tất cả các địa chỉ vật lý tương ứng với các địa chỉ logic

Đơn vị quản lý bộ nhớ (Memory-Management Unit - MMU)

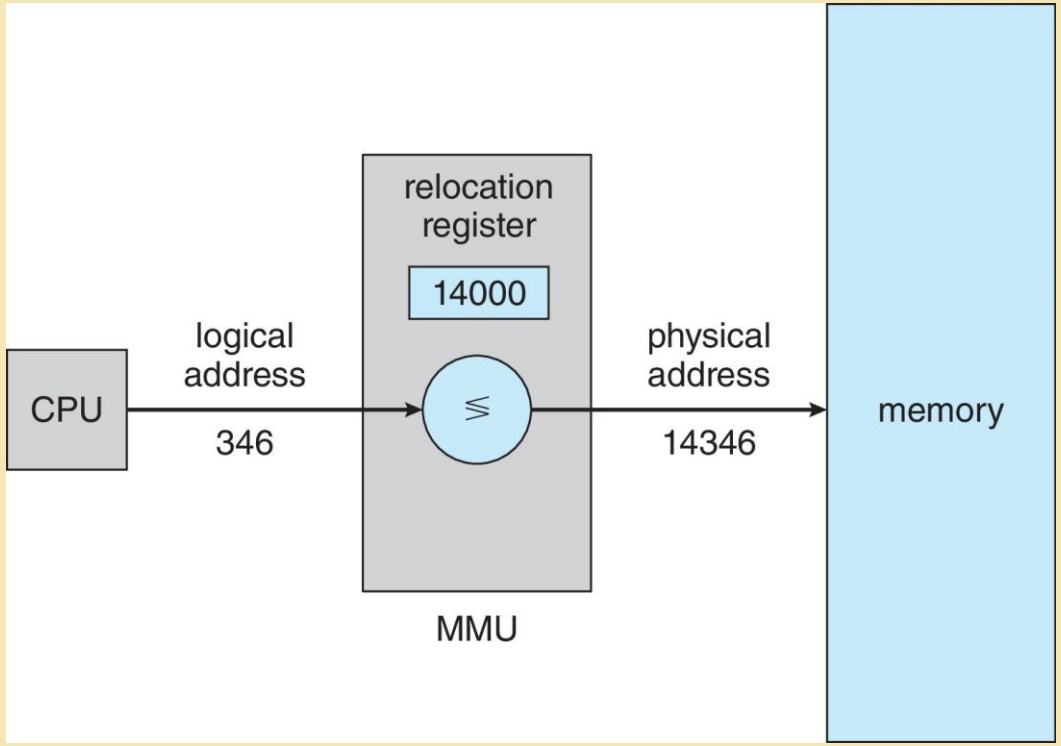
- Thiết bị phần cứng ánh xạ địa chỉ ảo tới địa chỉ vật lý tại thời điểm chạy



- Có nhiều phương thức ánh xạ

Đơn vị quản lý bộ nhớ (Memory-Management Unit - MMU)

- Minh họa việc ánh xạ với lược đồ MMU đơn giản, khái quát của lược đồ thanh ghi nền
- Thanh ghi nền **(base register)** lúc này được gọi là thanh ghi tái định vị **(relocation register)**
- Giá trị trong thanh ghi tái định vị được thêm vào mỗi địa chỉ được tạo ra bởi tiến trình người dùng tại thời điểm nó được gửi đến bộ nhớ
- Chương trình của người sử dụng chỉ thao tác trên các địa chỉ ảo, không bao giờ nhìn thấy các địa chỉ vật lý thực sự
  - Kết buộc tại thời điểm xử lý xảy ra khi địa chỉ tham chiếu được xác định vị trí trong bộ nhớ
  - Địa chỉ logic liên kết với địa chỉ vật lý



Nạp động (Dynamic loading)

Toàn bộ chương trình và dữ liệu của một tiến trình phải nằm trong bộ nhớ vật lý để thực thi → kích thước của một tiến trình đã bị giới hạn bởi kích thước của bộ nhớ vật lý.

Để sử dụng không gian bộ nhớ tốt hơn, sử dụng nạp động. Với nạp động:

- Thủ tục không được nạp cho đến khi nó được gọi
- Thủ tục không sử dụng thì không bao giờ được nạp
- Tất cả các thủ tục được giữ trên đĩa theo định dạng nạp có thể tái định vị
- Hữu ích khi số lượng mã lớn cần xử lý những trường hợp xảy ra không thường xuyên
- Không đòi hỏi hỗ trợ đặc biệt từ hệ điều hành
  - Thực thi thông qua thiết kế chương trình
  - Hệ điều hành trợ giúp bằng cách cung cấp thư viện để thực hiện việc nạp động

Liên kết động (Dynamic Linking)

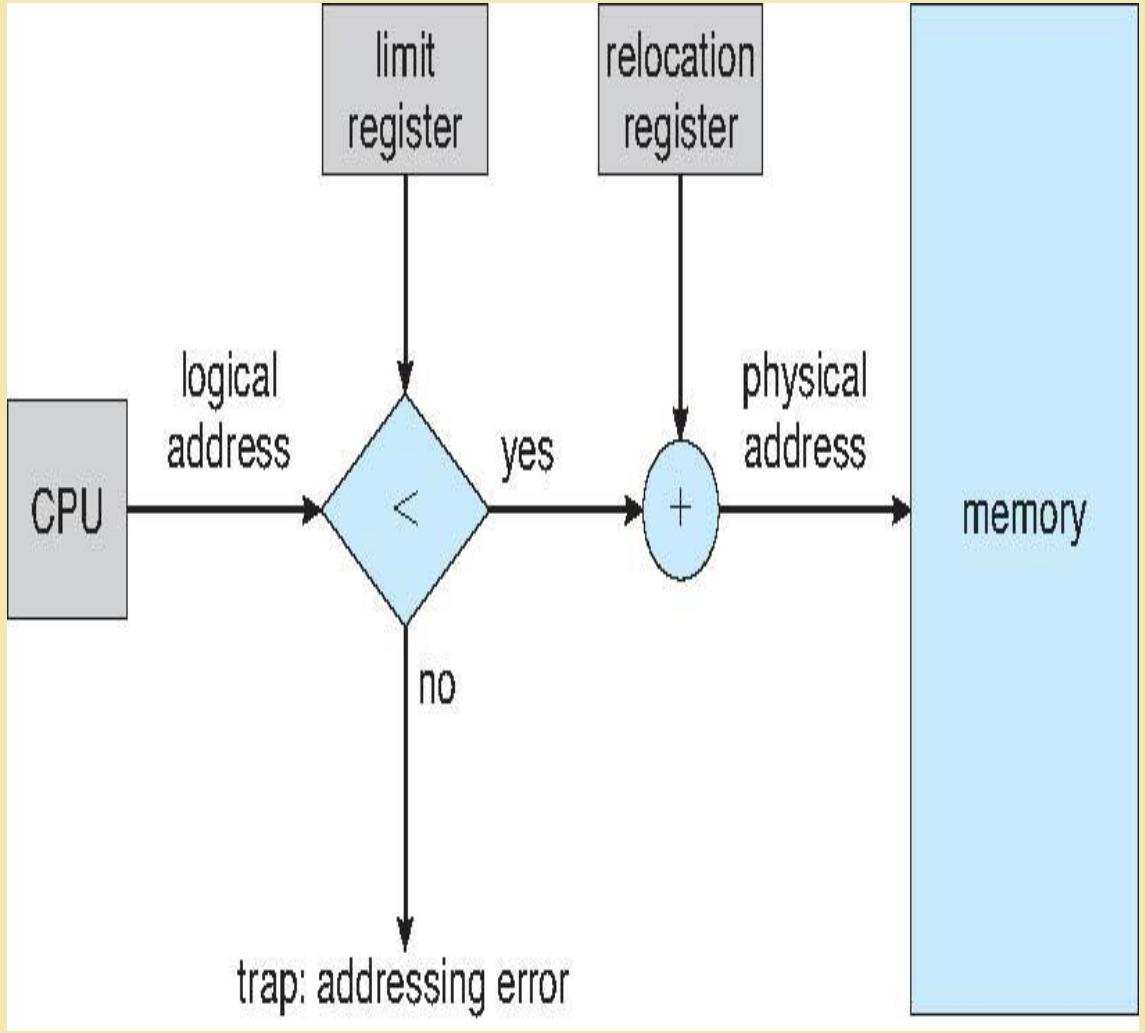
- Thư viện liên kết động (**Dynamically linked libraries (DLLs)**) là thư viện hệ thống được liên kết với chương trình người dùng khi chương trình chạy.
- Liên kết tĩnh (Static linking): thư viện hệ thống và mã chương trình được kết hợp thành ảnh chương trình nhị phân bởi trình nạp
- Liên kết động: liên kết được trì hoãn đến thời điểm thực thi
- Phần nhỏ của mã, **stub**, được sử dụng để định vị trí của thủ tục thư viện thường trú bộ nhớ thích hợp
- Stub tự thay thế nó bằng địa chỉ của thủ tục, và thực hiện các thủ tục
- Hệ điều hành kiểm tra xem thủ tục có nằm trong địa chỉ bộ nhớ của tiến trình hay không
  - Nếu không nằm trong không gian nhớ thì thêm vào
- Liên kết động đặc biệt hữu ích cho các thư viện
- Hệ thống còn được gọi là thư viện chia sẻ

2. Cấp phát bộ nhớ liên tục

- Bộ nhớ chính thường được chia thành 2 phần:
  - Hệ điều hành thường trú, thường được giữ trong bộ nhớ thấp với vector ngắt
  - Các tiến trình người dùng được giữ trong bộ nhớ cao
- Bộ nhớ chính phải hỗ trợ cả hệ điều hành và các tiến trình người dùng → Cần cấp phát bộ nhớ theo cách hiệu quả nhất
- Cấp phát bộ nhớ liên tục (Contiguous Memory Allocation) là một trong những phương pháp có rất sớm. Mỗi tiến trình được chứa trong một vùng nhớ liên tục duy nhất, tiếp giáp với vùng nhớ chứa tiến trình tiếp theo

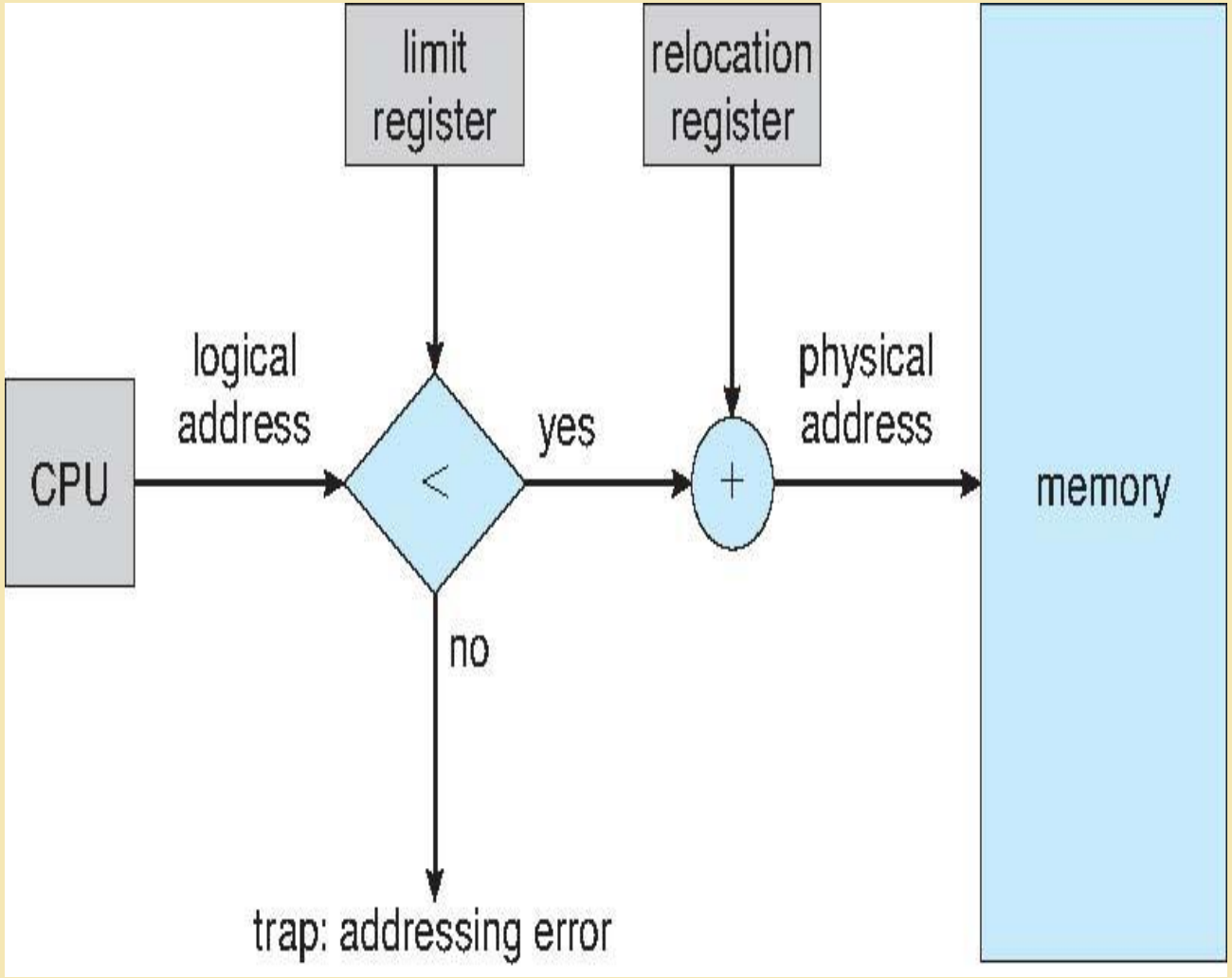
Bảo vệ bộ nhớ trong cấp phát bộ nhớ liên tục

- Các thanh ghi tái định vị được sử dụng để bảo vệ các tiến trình người dùng từ các tiến trình khác và từ sự thay đổi mã và dữ liệu hệ điều hành
  - Thanh ghi nền chứa đựng giá trị địa chỉ vật lý nhỏ nhất
  - Thanh ghi giới hạn chứa đựng vùng địa chỉ logic - mỗi địa chỉ logic phải nhỏ hơn thanh ghi giới hạn
  - MMU ánh xạ địa chỉ logic một cách tự động



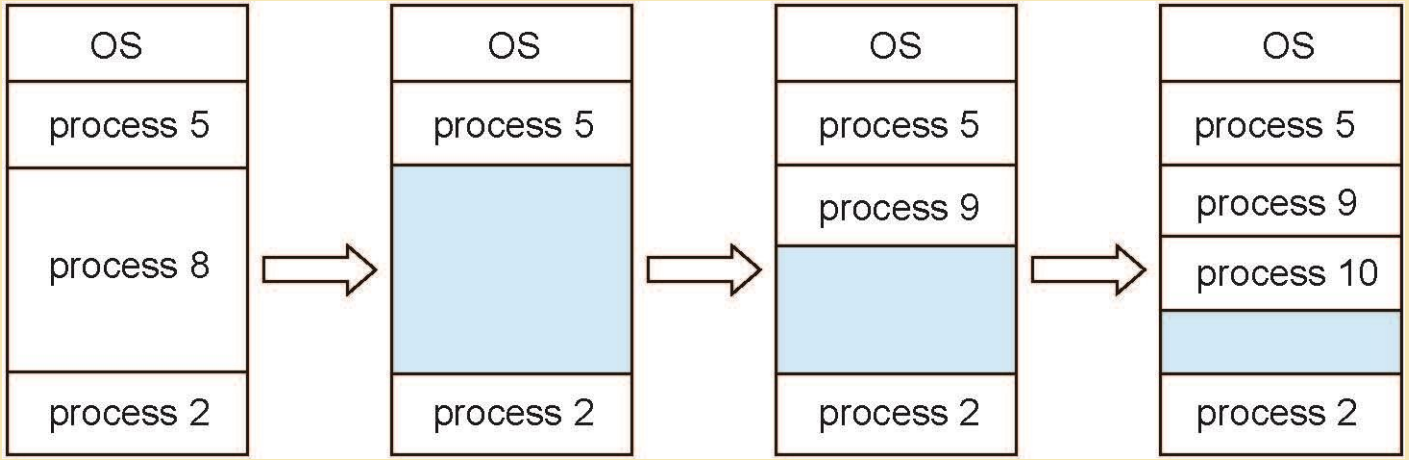
Bảo vệ bộ nhớ trong cấp phát bộ nhớ liên tục

- Ví dụ: relocation register = 100040; limit register = 74600
- Mỗi địa chỉ logic hợp lệ phải nằm trong vùng được xác định bởi limit register



Cấp phát bộ nhớ liên tục – variable partition

- Mức độ đa chương bị giới hạn bởi số phần
- Phân vùng thay đổi (**variable-partition**) được sắp xếp theo kích thước để hiệu quả hơn (sắp xếp kích thước theo nhu cầu của tiến trình)
- **Hole** – khối nhớ có sẵn; các **hole** kích thước khác nhau nằm rải rác khắp bộ nhớ
- Khi một tiến trình đến, nó được cấp phát bộ nhớ từ **hole** đủ lớn để chứa nó
- Tiến trình đang thoát ra giải phóng phân vùng của nó, phân vùng trống liền kề được kết hợp
- Hệ điều hành duy trì thông tin về:
  - Các phân vùng được cấp
  - Các phân vùng trống (**hole** - hố nhớ)



Vấn đề cấp phát động

Làm thế nào để thỏa mãn một yêu cầu với kích thước n từ danh sách các hố nhớ trống?

- First-fit: cấp phát hố nhớ đầu tiên đủ lớn
- Best-fit: cấp phát hố nhớ nhỏ nhất đủ lớn; phải tìm toàn bộ danh sách, trừ khi được sắp theo kích cỡ
  - Sinh ra các hố nhớ còn lại nhỏ nhất
- Worst-fit: cấp phát hố nhớ lớn nhất; cũng phải tìm toàn danh sách
  - Sinh ra các hố nhớ còn lại lớn nhất

First-fit và Best-fit tốt hơn Worst-fit về các khoản: tốc độ và việc sử dụng lưu trữ

Sự phân mảnh

- Phân mảnh bên ngoài (External Fragmentation) – tổng không gian bộ nhớ tồn tại thỏa mãn một yêu cầu, nhưng nó không liên tục
- Phân tích First-fit cho thấy rằng cứ N khối được cấp phát, 0.5N khối nữa bị mất vì phân mảnh
  - 1/3 bộ nhớ có thể không được sử dụng → quy tắc 50%
- Phân mảnh bên trong (Internal Fragmentation) - Bộ nhớ được cấp phát có thể lớn hơn một ít so với bộ nhớ yêu cầu; Sự khác biệt kích thước này là phần bộ nhớ bên trong một phân vùng, nhưng không được sử dụng

Giải quyết sự phân mảnh

- Giảm phân mảnh bên ngoài bằng cách nén bộ nhớ (compaction)
  - Xáo trộn (bố trí lại) nội dung bộ nhớ để đặt tất cả bộ nhớ trống vào cùng một khối lớn
  - Phương pháp nén chỉ có thể xảy ra nếu tái định vị là động, và được thực hiện tại thời điểm thực thi
  - Vấn đề I/O:
    - Chốt công việc trong bộ nhớ trong khi nó được tham gia vào I/O
    - Chỉ thực hiện I/O ở bên trong bộ đệm hệ điều hành
- Kho lưu trữ (backing store) cũng có cùng vấn đề phân mảnh

3. Phân trang (Paging)

- Không gian địa chỉ vật lý của một tiến trình có thể không liên tục; tiến trình được cấp phát bộ nhớ vật lý bất cứ khi nào có sẵn
  - Tránh phân mảnh bên ngoài
  - Tránh vấn đề thay đổi kích thước đoạn bộ nhớ (memory chunks)
- Chia bộ nhớ vật lý thành các khối nhớ có kích thước cố định, gọi là khung **(Frames)**
  - Kích thước là lũy thừa của 2, phụ thuộc vào kiến trúc máy tính (thông thường 4KB → 1 GB trên trang)
- Chia bộ nhớ logic thành các khối nhớ có kích thước bằng nhau, gọi là trang **(Pages)**
- Giữ tất cả các khung trang trống
- Để chạy một chương trình kích thước n trang, cần tìm n khung trang trống và nạp chương trình
- Thiết lập một bảng trang **(page table)** để chuyển đổi các địa chỉ logic thành địa chỉ vật lý
- Kho lưu trữ cũng chia thành các trang
- Vẫn còn phân mảnh bên trong

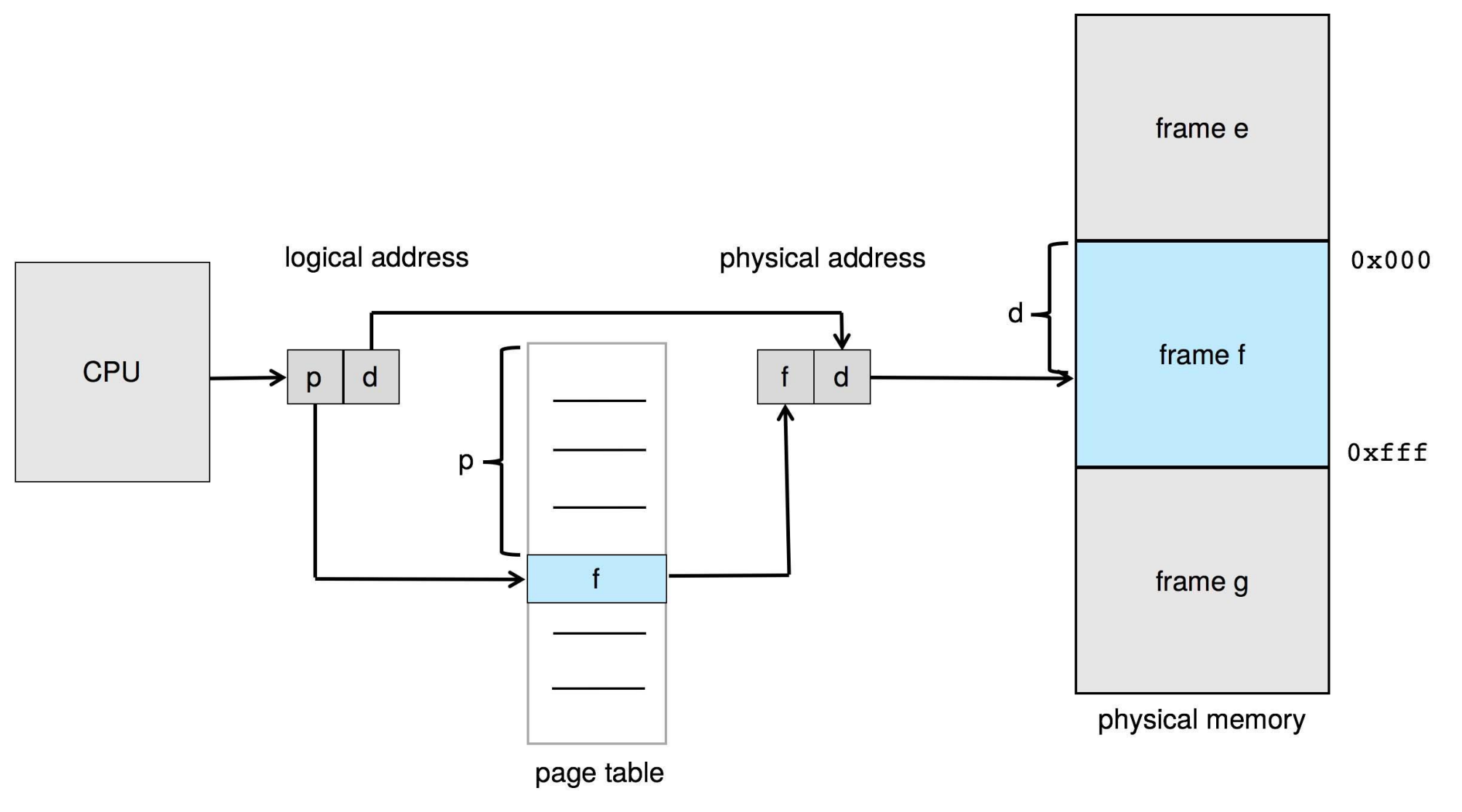
Lược đồ chuyển đổi địa chỉ

- Địa chỉ tạo ra bởi CPU được chia thành:
  - Số trang p (page number) – được sử dụng như một chỉ mục trong bảng trang, nơi chứa địa chỉ thanh ghi nền của mỗi trang trong bộ nhớ vật lý
  - Độ lệch trang d (page offset) – kết hợp với địa chỉ thanh ghi nền để xác định địa chỉ bộ nhớ vật lý mà được gởi tới đơn vị nhớ

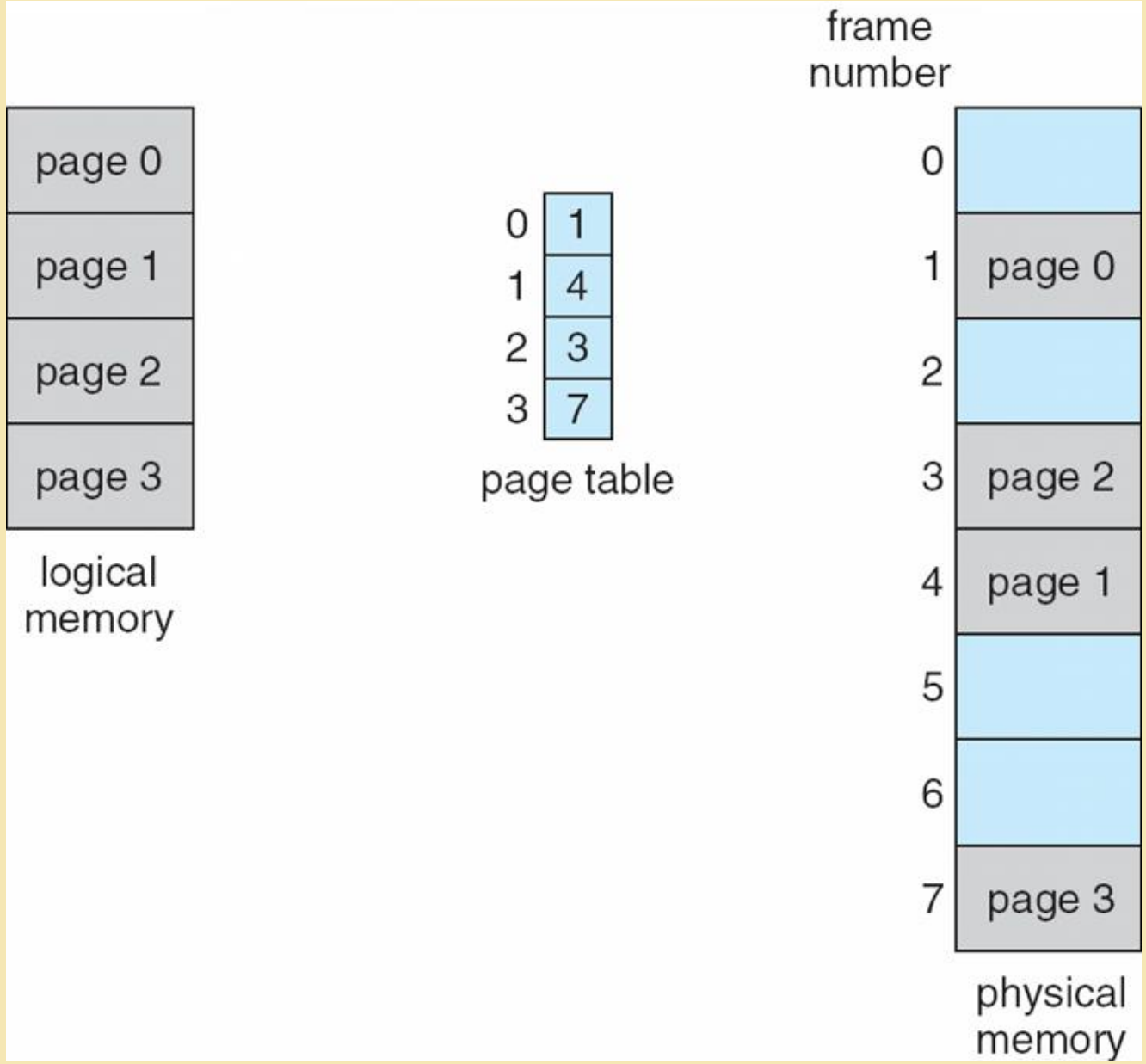
page number	page offset
p	d
m -n	n

  - Với kích thước không gian địa chỉ logic được cho 2^m và kích thước trang là 2^n

Phần cứng phân trang



Mô hình phân trang của bộ nhớ vật lý và logic

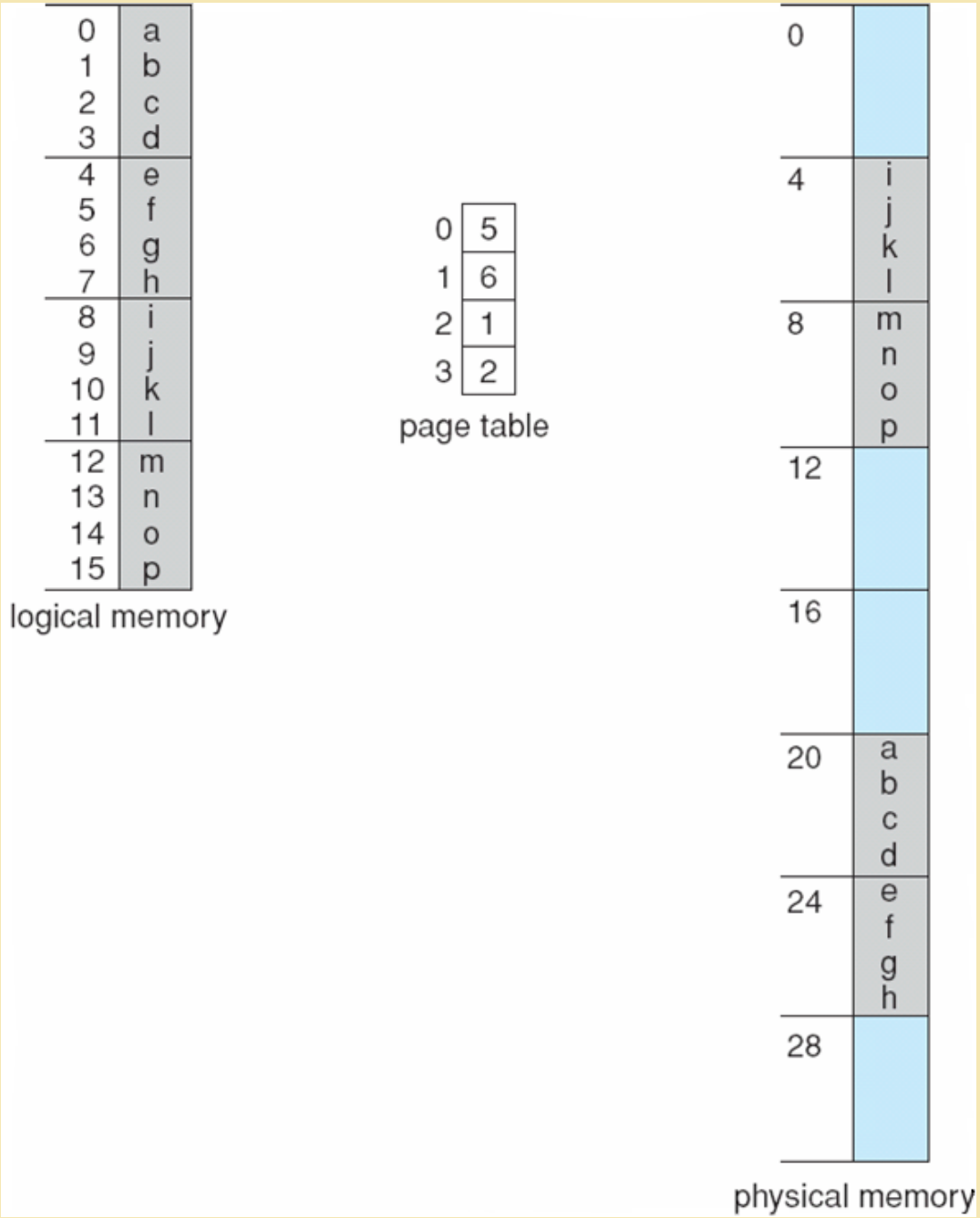


Ví dụ phân trang

- *Địa chỉ logic: n*=2 và *m*=4

Kích thước trang 4-byte

Bộ nhớ vật lý 32-byte (8 trang)

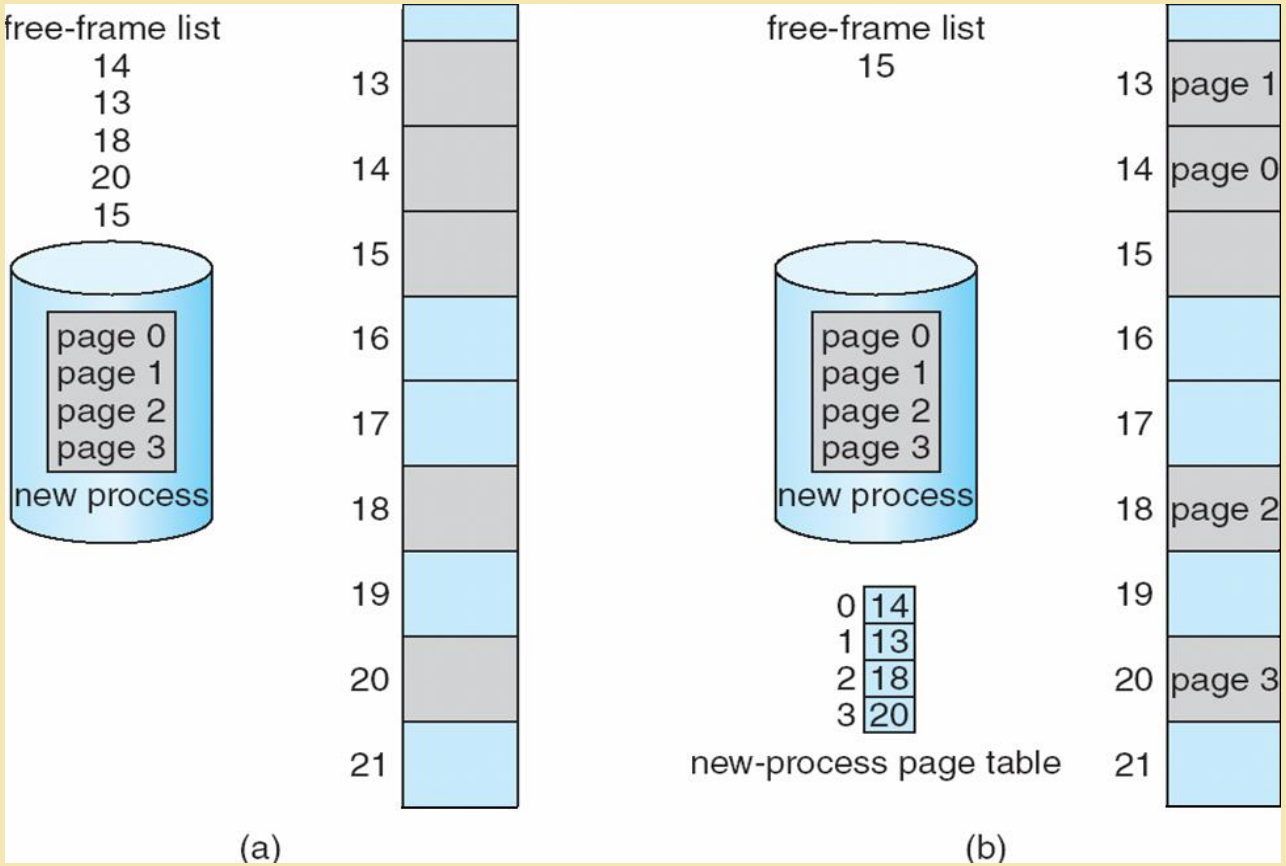


Phân trang - Tính toán phân mảnh bên trong

- Kích thước trang = 2,048 bytes
- Kích thước tiến trình = 72,766 bytes
- 35 trang + 1,086 bytes
- Phân mảnh bên trong của 2,048 - 1,086 = 962 bytes
- Trường hợp phân mảnh xấu nhất = 1 frame – 1 byte
- Phân mảnh trung bình = 1/2 kích thước khung
- Kích thước khung nhỏ như mong muốn?
- Nhưng mỗi mục nhập của bảng trang đều chiếm bộ nhớ
- Kích thước trang tăng theo thời gian
  - Solaris hỗ trợ 2 kích thước trang: 8KB và 4MB

Các khung trống

- Khi tiến trình đến hệ thống để thực thi, kích thước của nó được chia thành các trang.
- Trang đầu của tiến trình được nạp vào một khung, số khung được thêm vào bảng trang cho tiến trình này



Trước cấp phát Sau cấp phát

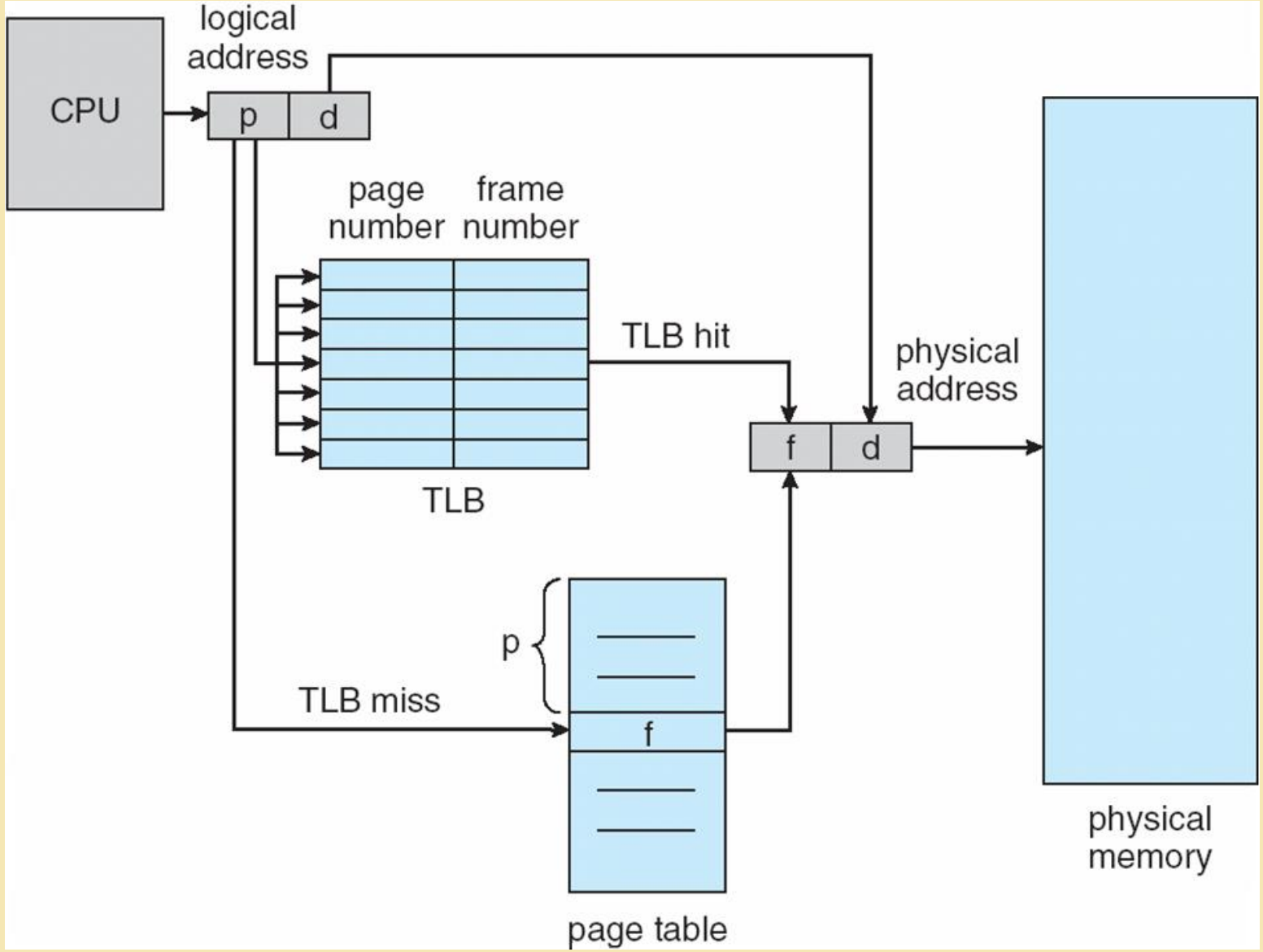
Cài đặt bảng trang

Theo nhiều cách:

1) Sử dụng tập thanh ghi tốc độ cao để chuyển đổi địa chỉ
   - Phù hợp khi bảng trang có kích thước nhỏ
   - Tốn thời gian chuyển ngữ cảnh nhiều
2) Giữ bảng trang trong bộ nhớ chính

- Bảng trang được giữ trong bộ nhớ chính
  - Thanh ghi nền bảng trang (Page-table base register - PTBR) trỏ tới bảng trang
  - Thanh ghi chiều dài bảng trang (Page-table length register – PTLR) cho biết kích thước bảng trang
- Trong lược đồ này, mỗi truy cập dữ liệu/chỉ thị đòi hỏi 2 truy cập bộ nhớ
  - Một cho bảng trang và một cho dữ liệu/chỉ thị
- Vấn đề 2 truy cập bộ nhớ có thể được giải quyết bằng cách sử dụng một bộ đệm tham chiếu nhanh đặc biệt, được gọi là bộ nhớ kết hợp (**associative memory**, hoặc **translation look-aside buffers** (**TLBs**))

Phân trang với TLB



Translation look-aside buffers (TLBs)

- Một số TLBs lưu trữ định danh không gian địa chỉ (**address-space identifiers** (**ASIDs**)) trong mỗi mục TLB - nhận diện duy nhất mỗi tiến trình để cung cấp sự bảo vệ không gian địa chỉ cho tiến trình đó
  - Nếu không cần phải xóa sạch TLB tại mỗi lần chuyển ngữ cảnh
- Các TLBs thường nhỏ (64 tới 1024 mục)
- Trên một **TLB miss** (số trang không có trong TLB), giá trị được nạp vào TLB cho truy cập nhanh hơn vào lần kế tiếp
  - Các chính sách thay thế phải được xem xét
  - Một số mục có thể được **wired down** (chúng không thể bị gỡ bỏ khỏi TLB) để truy cập nhanh thường xuyên

Phần cứng - Bộ nhớ kết hợp

- Bộ nhớ kết hợp – tìm kiếm song song

Page #	Frame #

- Chuyển đổi địa chỉ (p,d)
  - Nếu p nằm trong thanh ghi kết hợp, lấy frame # ra.
  - Nếu không thì lấy frame # từ bảng trang trong bộ nhớ.

Thời gian truy cập hiệu quả

- Hit ratio (Tỷ lệ truy cập) - là tỷ lệ phần trăm số lần mà một số trang được tìm thấy trong TLB.
- 80% hit ratio, có nghĩa là đã tìm thấy 80 % trang mong đợi trong TLB.
- Giả sử có 10 nanoseconds để truy cập bộ nhớ
  - Nếu tìm thấy trang mong đợi trong TLB, thì một ánh xạ bộ nhớ mất 10 ns
  - Ngược lại cần 2 lần truy cập bộ nhớ, mất 20 ns
- **Effective Access Time (EAT)**

  EAT = 0.80 x 10 + 0.20 x 20 = 12 ns

  →Thời gian truy cập chậm 20%
- Với hit ratio = 99%

  EAT = 0.99 x 10 + 0.01 x 20 = 10.1 ns

  →Thời gian truy cập chỉ chậm 1%

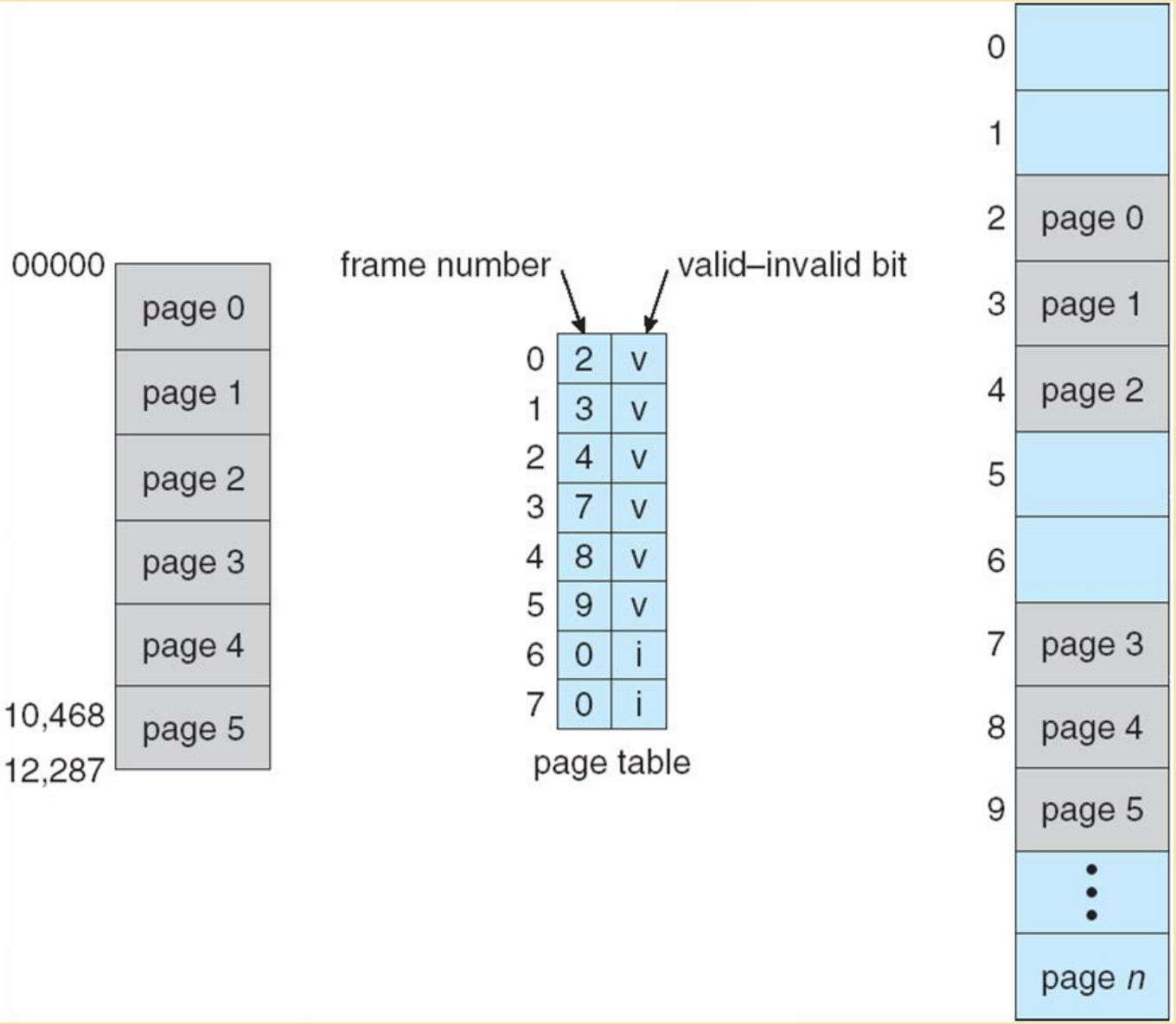
Bảo vệ bộ nhớ

- Bảo vệ bộ nhớ thực hiện bằng cách liên kết bit bảo vệ với mỗi khung để cho biết là truy cập read-only hoặc truy cập read-write được phép
  - Cũng có thể thêm nhiều bit để chỉ ra trang chỉ thực thi,…
- Bit hợp lệ - không hợp lệ (valid-invalid Bit) được gắn với mỗi mục nhập trong bảng trang
  - "valid" cho biết rằng: trang được liên kết nằm trong không gian địa chỉ logic của tiến trình, và do đó là một trang hợp lệ
  - "invalid" cho biết rằng: trang không nằm trong không gian địa chỉ logic của tiến trình
  - Hoặc sử dụng thanh ghi chiều dài bảng trang (**page-table length register** (**PTLR**))
- Bất kỳ vi phạm nào sẽ dẫn đến một bẫy lỗi (trap) cho nhân hệ điều hành.

Bit hợp lệ hoặc không hợp lệ trong bảng trang

<u>Ví dụ:</u>

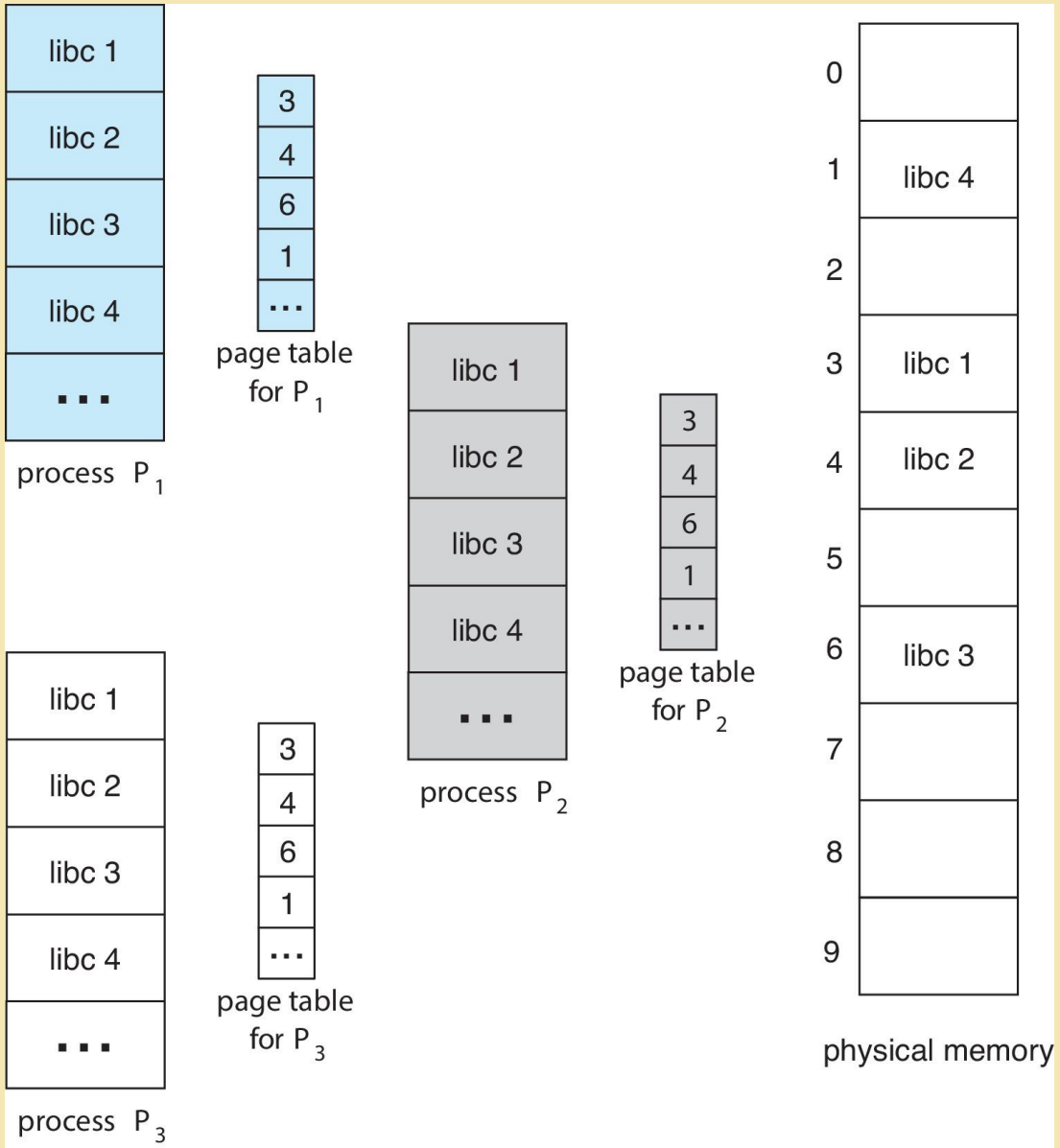
- Hệ thống có không gian địa chỉ 14-bit (0 tới 16383)
- Một chương trình chỉ sử dụng địa chỉ từ 0 tới 10468
- Kích thước trang là 2KB



Chia sẻ trang

- Mã chia sẻ
  - Một bản sao của mã chỉ đọc (**reentrant code**) được chia sẻ giữa các tiến trình (trình soạn thảo, trình biên dịch, hệ thống cửa sổ,…)
  - Tương tự như đa tiểu trình chia sẻ cùng không gian tiến trình
  - Cũng hữu ích cho truyền thông liên tiến trình khi việc chia sẻ các trang đọc-ghi được cho phép
- Mã và dữ liệu riêng
  - Mỗi tiến trình giữ một bản sao riêng của mã và dữ liệu
  - Các trang với mã và dữ liệu riêng có thể xuất hiện ở bất cứ nới nào trong không gian địa chỉ logic

Ví dụ về chia sẻ trang

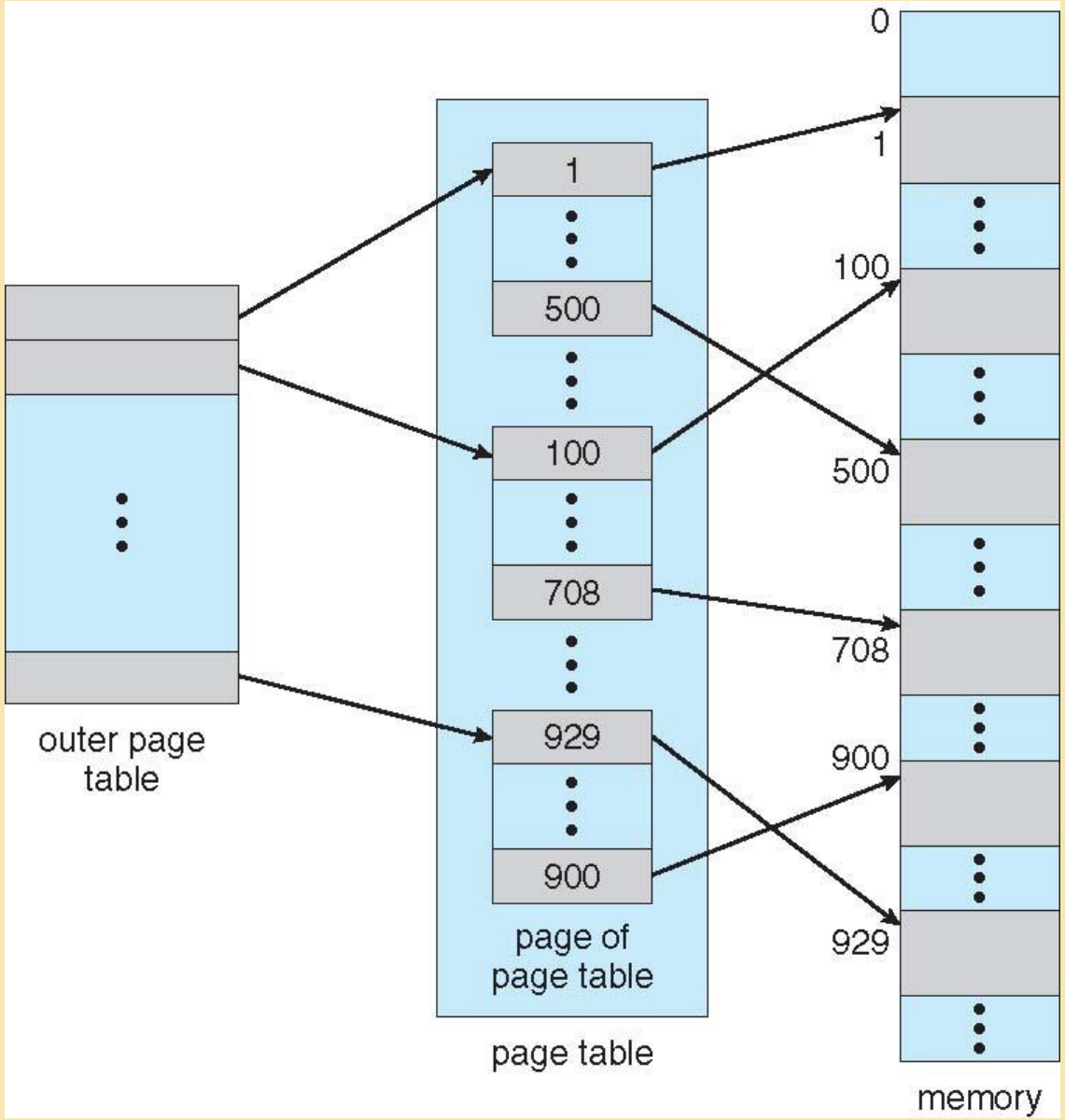


4. Cấu trúc bảng trang

- Cấu trúc bộ nhớ cho phân trang có thể có rất nhiều bằng cách sử dụng các phương đơn giản
  - Xem xét một không gian địa chỉ logic 32-bit trên các máy tính hiện đại
  - Kích thước trang: 4KB (2^{12})
  - Bảng trang sẽ có 1 triệu mục nhập ($2^{32}/2^{12}$)
  - Nếu mỗi mục nhập là 4 bytes → 4MB không gian địa chỉ vật lý/ bộ nhớ cho 1 bảng trang
    - Không muốn cấp phát bảng trang liên tục trong bộ nhớ chính
- Một giải pháp đơn giản là chia bảng trang thành các đơn vị nhỏ hơn
  - Bảng trang phân cấp
  - Bảng trang băm
  - Bảng trang nghịch đảo

Bảng trang phân cấp

- Chia nhỏ không gian địa chỉ logic thành nhiều bảng trang
- Kỹ thuật đơn giản là bảng trang 2 cấp
- Sau đó phân trang bảng trang



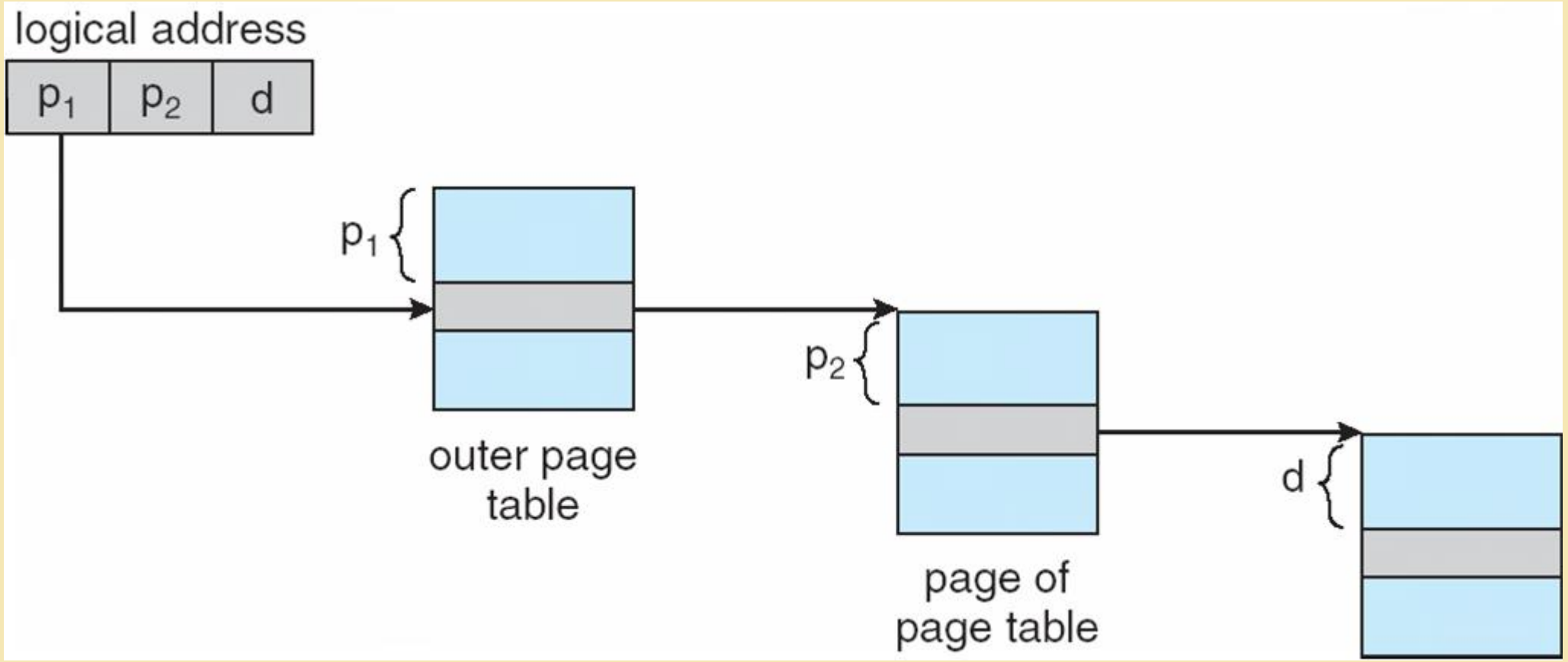
Ví dụ phân trang 2 cấp

- Một địa chỉ logic (trên máy 32bit với kích thước trang 1K) được chia thành:
  - Một số trang bao gồm 22bit
  - Một độ lệch trang bao gồm 10bit
- Vì bảng trang được phân trang, nên số trang được chia thêm nữa thành:
  - 10 bit số trang
  - 12 bit độ lệch trang
- Do vậy, địa chỉ logic là như sau:

page number		page offset
p_1	p_2	d
10	10	12

- ở đó: p1 là chỉ mục vào bảng trang bên ngoài, và p2 là độ lệch trong trang của bảng trang bên trong
- Được biết như là **forward-mapped page table**

Lược đồ chuyển đổi địa chỉ



Không gian địa chỉ logic 64-bit

- Lược đồ phân trang 2 cấp cũng chưa đầy đủ
- Nếu kích thước trang là 4KB (2^{12})
  - Thì bảng trang có 2^{52} mục nhập
  - Nếu lược đồ 2 cấp, thì bảng trang bên trong chứa 2^{10} các mục nhập 4byte
  - Địa chỉ sẽ như sau:

outer page	inner page	page offset
p_1	p_2	d
42	10	12

  - Bảng trang bên ngoài có 2^{42} mục nhập hoặc 2^{44}byte
  - Một giải pháp là cộng thêm một bảng trang bên ngoài thứ 2
  - Nhưng trong ví dụ sau, bảng trang bên ngoài thứ 2 vẫn chỉ có 2^{34}byte kích thước
    - Và có thể có 4 truy cập bộ nhớ đến một vị trí bộ nhớ vật lý

Lược đồ phân trang 3 cấp

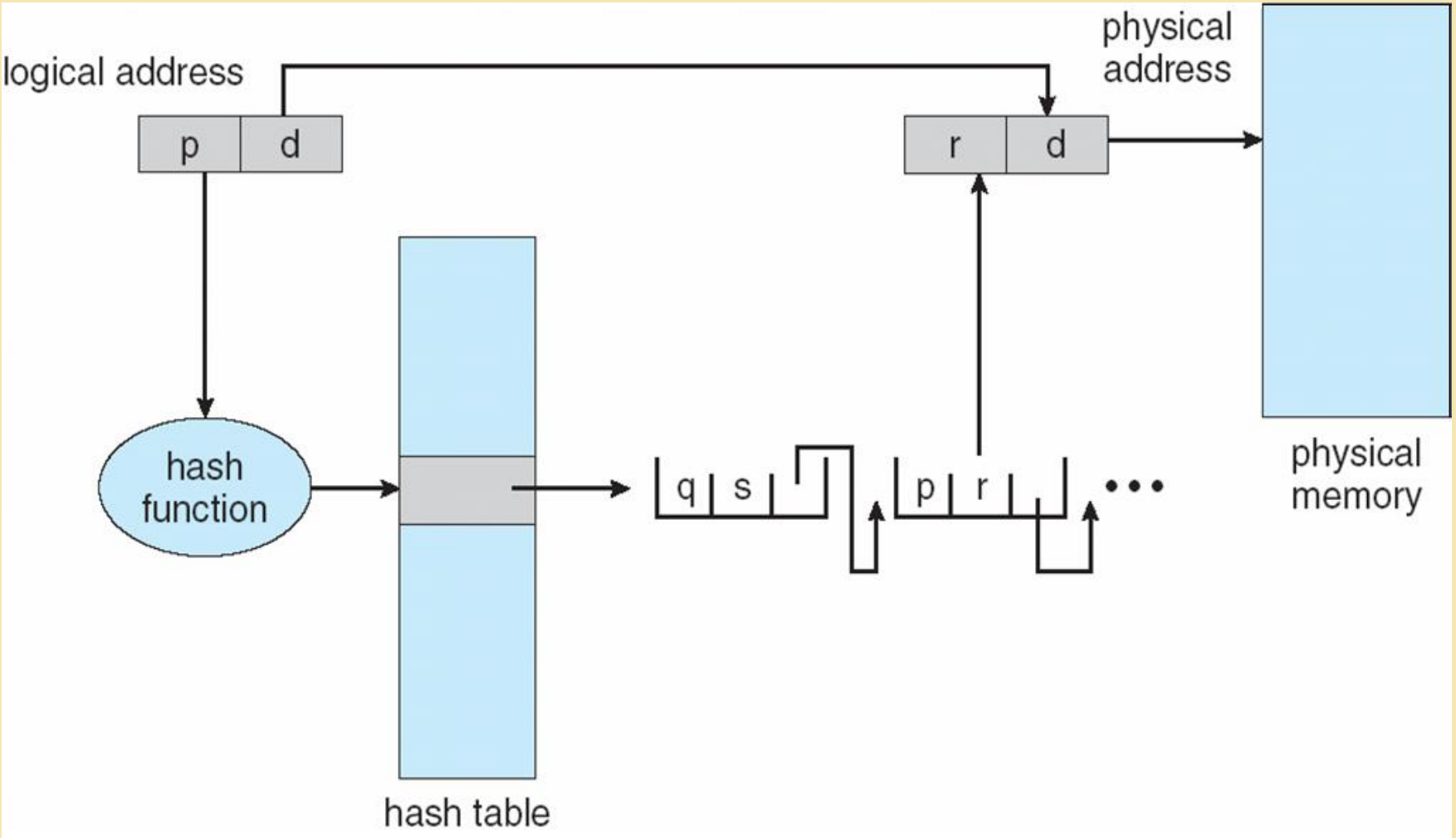
outer page	inner page	offset
p_1	p_2	d
42	10	12

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

Bảng trang băm

- Phổ biến trong không gian địa chỉ > 32-bit
- Số trang ảo được băm trong bảng trang
  - Bảng trang này chứa một chuỗi các phần tử băm tới cùng vị trí
- Mỗi phần tử chứa: (1) số trang ảo; (2) giá trị của khung trang ánh xạ và (3) một con trỏ tới phần tử kế tiếp
- Các số trang ảo được so sánh trong chuỗi này để tìm ra phần tử khớp
  - Nếu phần tử khớp được tìm thấy, khung trang vật lý tương ứng sẽ được trích xuất
- Dạng khác cho địa chỉ 64bit là bảng trang gom cụm
  - Tương tự như bảng trang băm, nhưng mỗi mục nhập tương ứng với nhiều trang (chẳng hạn 16) thay vì 1
  - Đặc biệt hữu ích cho các không gian địa chỉ **sparse** (các tham chiếu bộ nhớ không liên tục và rời rạc)

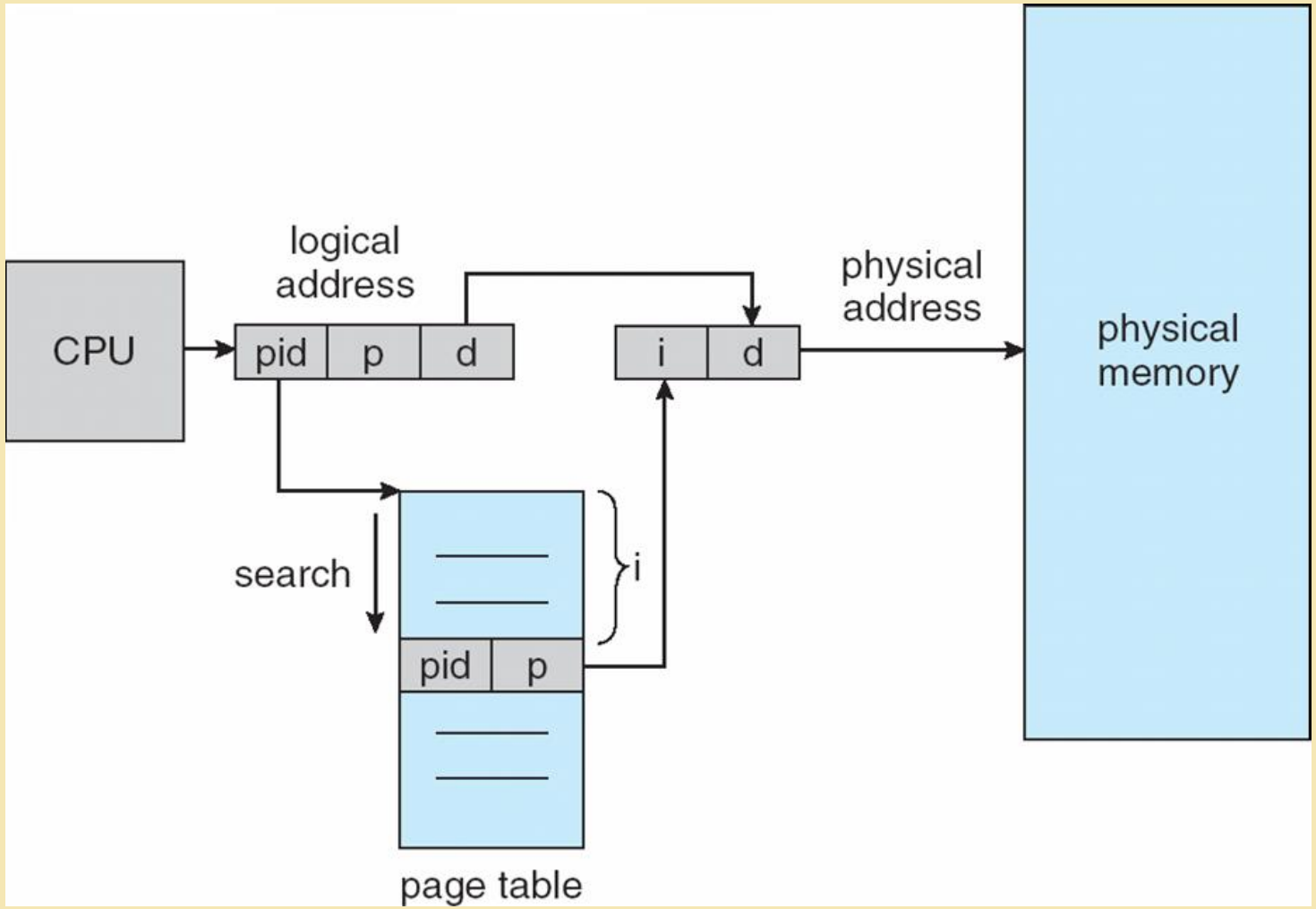
Bảng trang băm



Bảng trang nghịch đảo

- Thay vì mỗi tiến trình có một bảng trang và theo dõi tất cả các trang logic có thể, thì theo dõi tất cả các trang vật lý
- Một mục nhập cho mỗi trang bộ nhớ thực
- Mục nhập bao gồm địa chỉ ảo của trang được lưu trữ trong vị trí bộ nhớ thực đó với thông tin về tiến trình sở hữu trang đó
- Giảm bộ nhớ cần thiết để lưu trữ từng bảng trang, nhưng làm tăng thời gian cần thiết để tìm kiếm bảng khi một tham chiếu trang xuất hiện
- Sử dụng bảng băm để giới hạn việc tìm kiếm một hoặc nhiều nhất một vài mục nhập trong bảng
  - TLB có thể tăng tốc truy cập
- Nhưng làm thế nào để thực thi bộ nhớ chia sẻ?
  - Một ánh xạ địa chỉ ảo đến địa chỉ vật lý chia sẻ

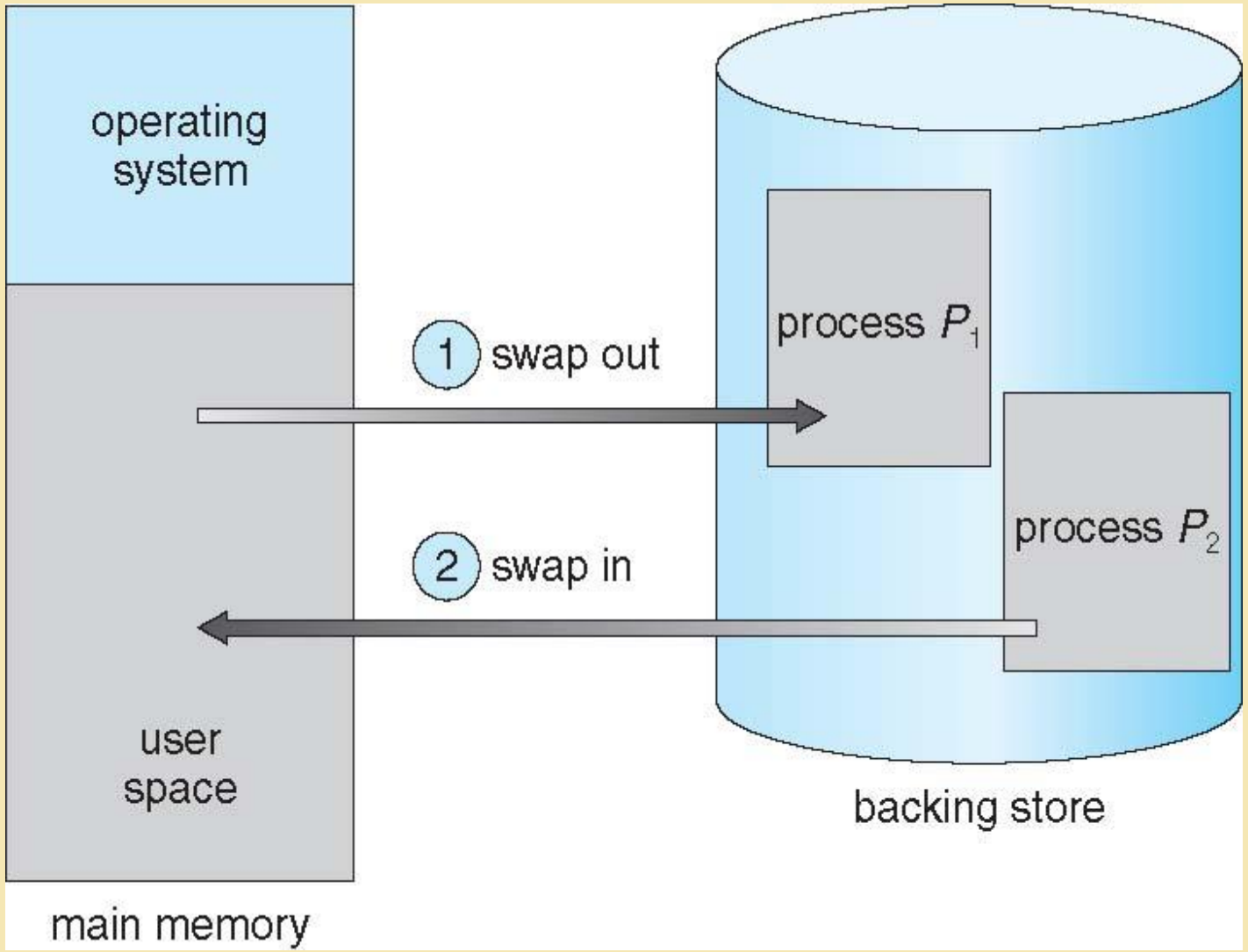
Kiến trúc bảng trang nghịch đảo



5. Hoán đổi tiến trình

Tổng không gian bộ nhớ vật lý của tiến trình có thể vượt quá bộ nhớ vật lý thật sự của hệ thống

→ Làm tăng mức độ đa chương cho hệ thống



5. Hoán đổi tiến trình

- Một tiến trình có thể được hoán đổi tạm thời ra khỏi bộ nhớ đến một backing store, và sau đó được đưa trở lại bộ nhớ để tiếp tục thực thi
  - Tổng không gian bộ nhớ vật lý của tiến trình có thể vượt quá bộ nhớ vật lý
- Backing store – đĩa tốc độ nhanh, đủ lớn để chứa các bản sao của tất cả các ảnh bộ nhớ cho tất cả người dùng; cung cấp truy cập trực tiếp tới ảnh bộ nhớ
- Chuyển ra, chuyển vào (Roll out, roll in) – hoán đổi sử dụng cho các giải thuật định thời theo độ ưu tiên; tiến trình có độ ưu tiên thấp hơn được hoán đổi ra ngoài để tiến trình có độ ưu tiên cao được nạp và thực thi
- Phần lớn thời gian hoán đổi là thời gian di chuyển của tiến trình; tổng thời gian di chuyển tỉ lệ thuận với lượng bộ nhớ được hoán đổi
- Hệ thống duy trì một hàng đợi sẵn sàng (ready queue) của các tiến trình đang sẵn sàng chạy, có ảnh bộ nhớ trên đĩa

Hoán đổi tiến trình

- Tiến trình được hoán đổi ra ngoài có cần hoán đổi vào cùng địa chỉ vật lý không?
- Phụ thuộc vào phương thức kết buộc địa chỉ
  - Cộng thêm xem xét việc chờ giải quyết I/O từ/đến không gian bộ nhớ tiến trình
- Phiên bản cập nhật của hoán đổi được tìm thấy trên nhiều hệ thống (như UNIX, Linux và Windows)
  - Hoán đổi thường bị vô hiệu hóa
  - Bắt đầu lại nếu số lượng bộ nhớ cấp phát nhiều hơn ngưỡng
  - Vô hiệu hóa lại khi nhu cầu bộ nhớ giảm xuống dưới ngưỡng

Thời gian chuyển ngữ cảnh trong hoán đổi

- Nếu các tiến trình tiếp theo được đặt vào CPU không có trong bộ nhớ, cần phải hoán đổi ra ngoài một tiến trình và hoán đổi vào trong tiến trình mục tiêu.
- Thời gian chuyển ngữ cảnh khi đó rất cao
- Tiến trình 100MB hoán đổi vào đĩa cứng với tốc độ truyền 50MB / giây
  - Thời gian hoán đổi ra ngoài 2000ms
  - Cộng với hoán đổi vào trong của tiến trình cùng kích thước
  - Tổng thời gian chuyển ngữ cảnh các thành phần hoán đổi 4000ms (4 giây)
- Có thể giảm nếu giảm kích thước của bộ nhớ hoán đổi – bằng cách biết bao nhiêu bộ nhớ thực sự đang được sử dụng
  - Thông qua lời gọi hệ thống để thông báo cho hệ điều hành bộ nhớ sử dụng

    `request_memory()` và `release_memory()`

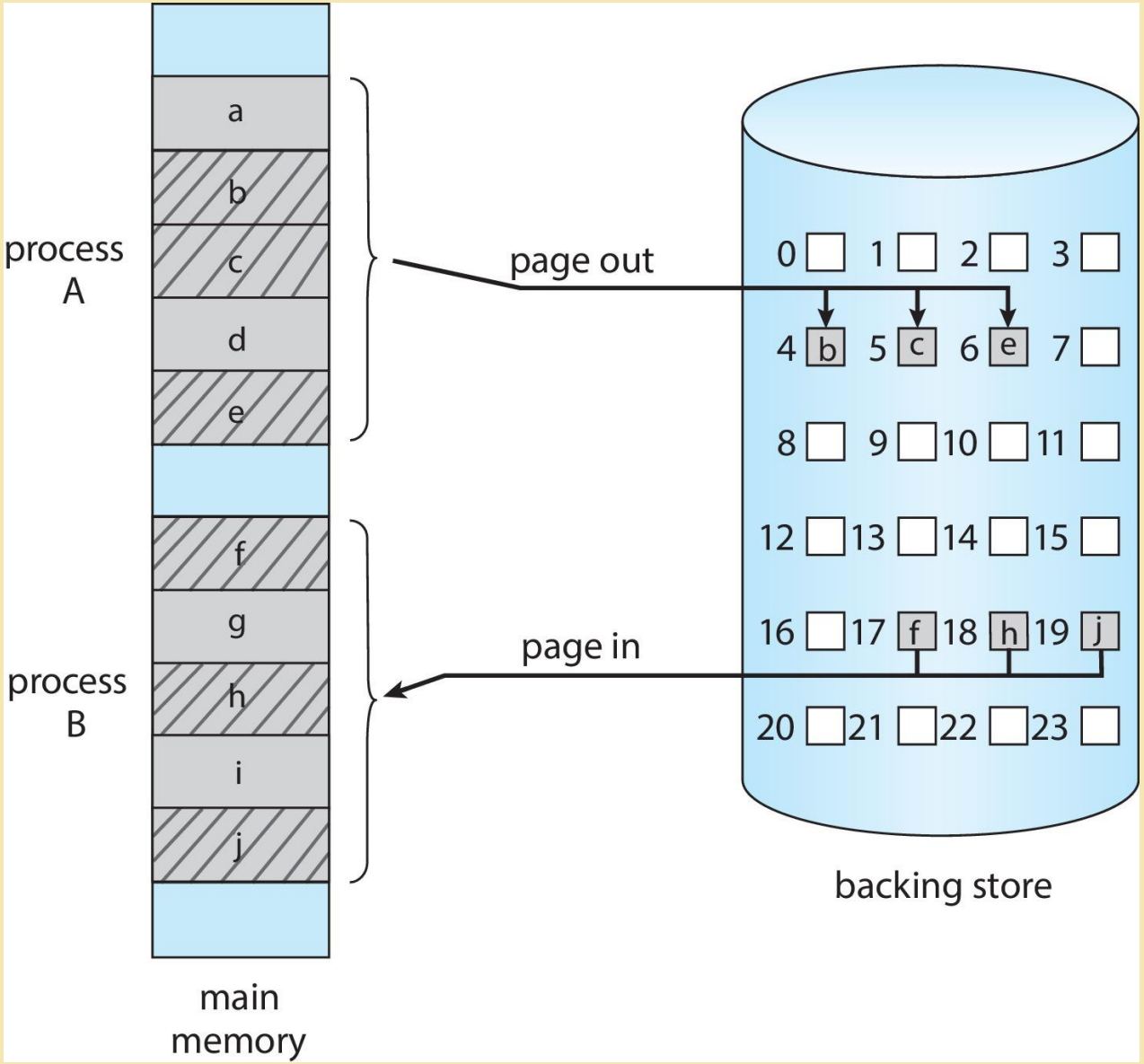
Thời gian chuyển ngữ cảnh trong hoán đổi

- Các ràng buộc khác cũng xét trên hoán đổi
  - Chờ I/O - không thể hoán đổi ra ngoài khi I/O tìm thấy tiến trình sai
  - Hoặc luôn luôn chuyển I/O tới không gian nhân, rồi tới thiết bị I/O
    - Được biết như **double buffering**, làm tăng thêm chi phí
- Hoán đổi tiêu chuẩn không được sử dụng trong các hệ điều hành hiện đại
  - Nhưng phiên bản cập nhật phổ biến
    - Hoán đổi chỉ thực hiện khi bộ nhớ trống quá thấp

Hoán đổi trên hệ thống di động

- Không hỗ trợ
  - Dựa vào bộ nhớ Flash
    - Số không gian nhỏ
    - Số vòng ghi giới hạn
    - Thông lượng thấp giữa bộ nhớ flash và CPU trên nền tảng di động
- Thay vào đó, sử dụng các phương pháp khác để giải phóng bộ nhớ khi thấp
  - iOS yêu cầu ứng dụng tự nguyện từ bỏ bộ nhớ được cấp phát
    - Dữ liệu chỉ đọc được xóa khỏi hệ thống và sau đó tải lại từ bộ nhớ flash nếu cần.
    - Lỗi giải phóng có thể dẫn đến sự chấm dứt ứng dụng.
  - Android chấm dứt các ứng dụng nếu bộ nhớ trống thấp, nhưng đầu tiên ghi lại trạng thái ứng dụng vào flash để khởi động lại nhanh.
  - Cả iOS và Android đều hỗ trợ ***phân trang***

Hoán đổi với phân trang



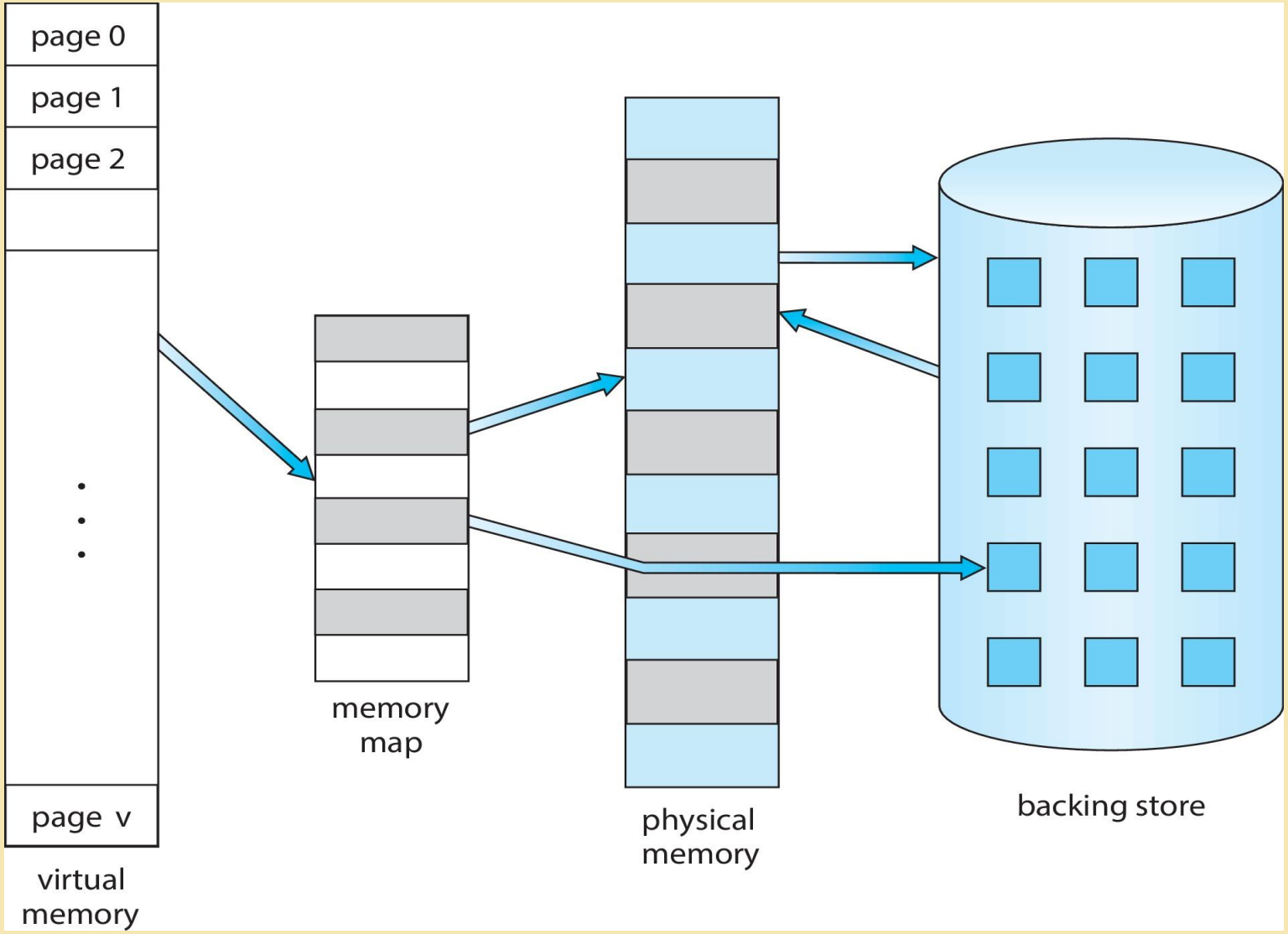
6. Bộ nhớ ảo

- Bộ nhớ ảo (Virtual memory) – tách bộ nhớ logic của người dùng khỏi bộ nhớ vật lý.
  - Chỉ 1 phần cần thiết của chương trình nằm trong bộ nhớ để thực thi
  - Do đó, không gian địa chỉ logic có thể lớn hơn nhiều so với không gian địa chỉ vật lý
  - Cho phép một số tiến trình chia sẻ không gian địa chỉ
  - Cho phép tạo tiến trình hiệu quả hơn
  - Nhiều chương trình chạy đồng thời
  - Cần ít I/O hơn để tải hoặc hoán đổi các tiến trình

Bộ nhớ ảo

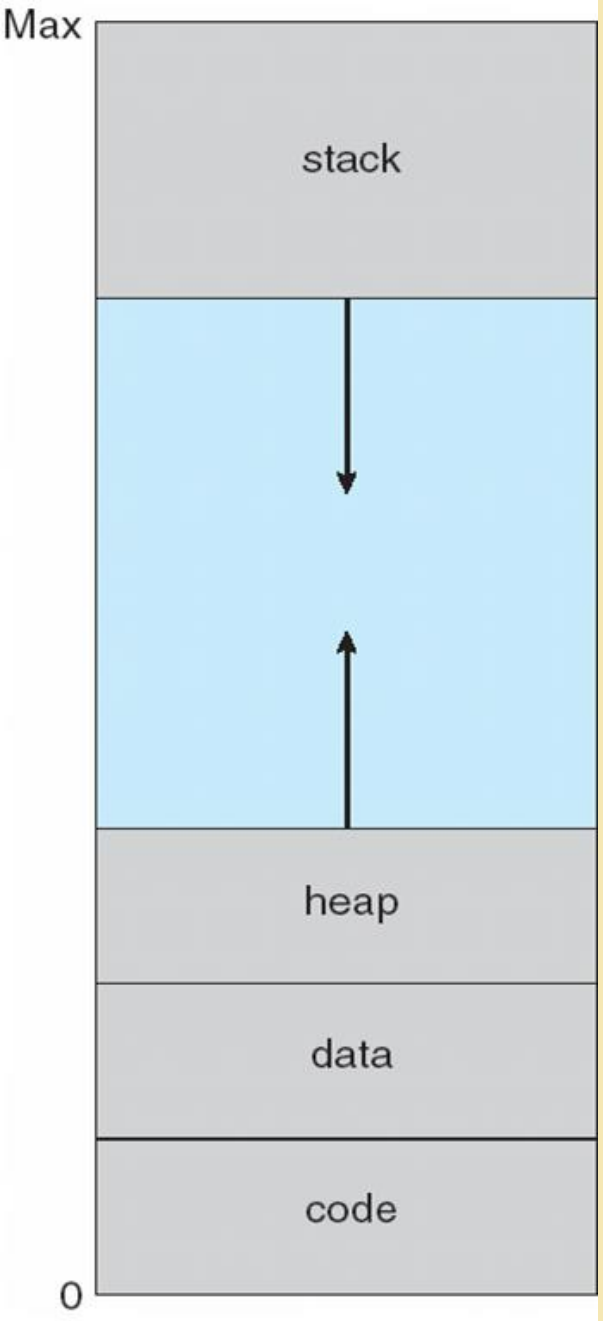
- Không gian địa chỉ ảo – khung nhìn logic về cách tiến trình được lưu trữ trong bộ nhớ
  - Thường bắt đầu từ địa chỉ 0, các địa chỉ liền nhau cho đến hết không gian
  - Trong khi đó, bộ nhớ vật lý được tổ chức theo các khung trang
  - MMU phải ánh xạ địa logic đến địa chỉ vật lý
- Bộ nhớ ảo có thể được thực thi thông qua
  - Phân trang theo yêu cầu (Demand paging)
  - Phân đoạn theo yêu cầu (Demand segmentation)

Bộ nhớ ảo lớn hơn bộ nhớ vật lý

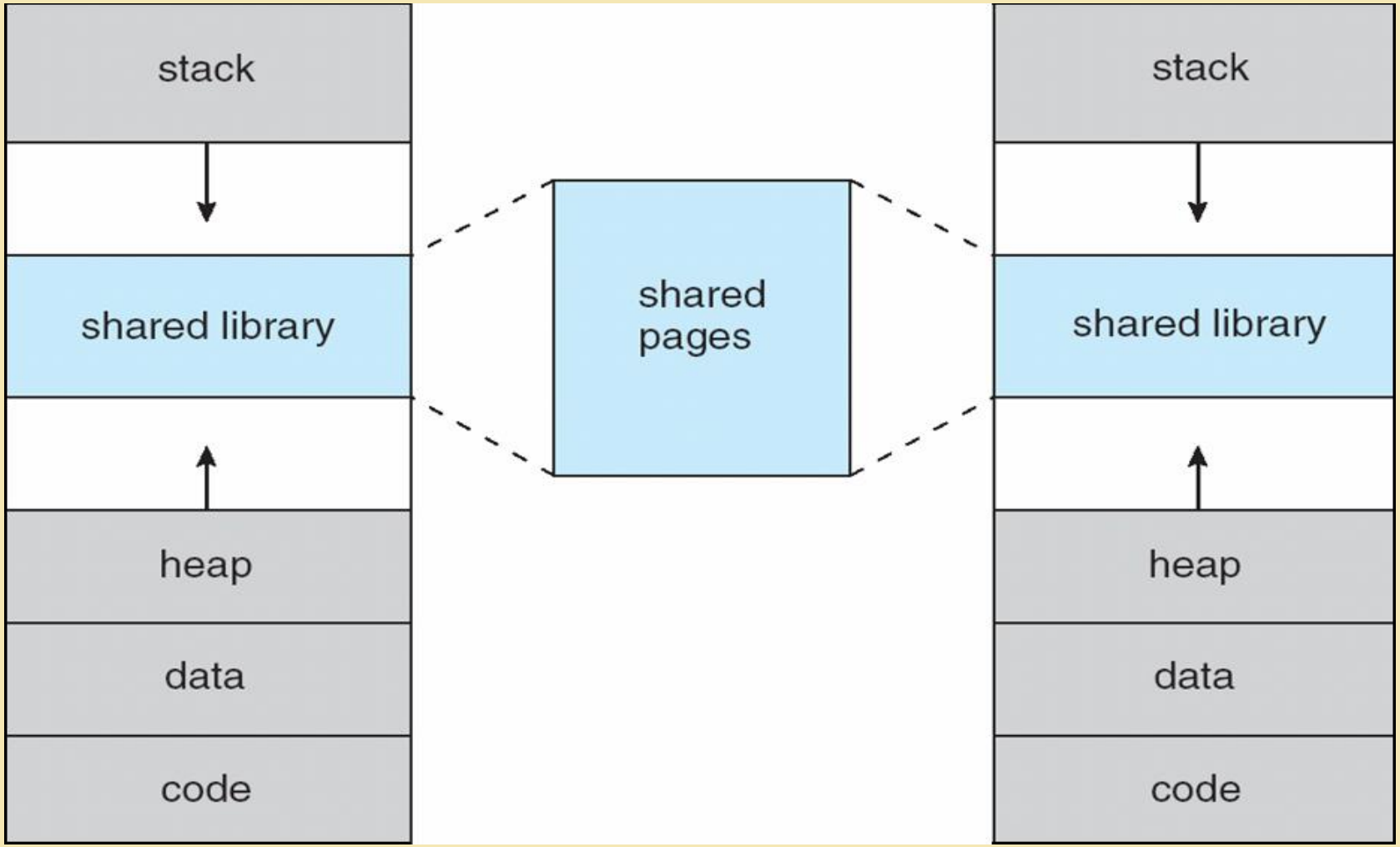


Không gian địa chỉ ảo

- Thiết kế không gian địa chỉ logic thường để ngăn xếp (stack) bắt đầu ở địa chỉ logic Max và tăng theo hướng "xuống", trong khi khối xếp (heap) tăng theo hướng "lên"
  - Tối đa hóa việc sử dụng không gian địa chỉ
  - Không gian địa chỉ không được sử dụng nằm giữa là vùng nhớ trống (hố nhớ - hole)
    - Không cần bộ nhớ vật lý cho đến khi khối xếp hoặc ngăn xếp tăng tới một trang mới.
- Thư viện hệ thống được chia sẻ thông qua việc ánh xạ vào không gian địa chỉ ảo
- Bộ nhớ được chia sẻ bằng cách ánh xạ các trang đọc-ghi vào không gian địa chỉ ảo
- Các trang có thể được chia sẻ trong fork(), tăng tốc việc tạo tiến trình

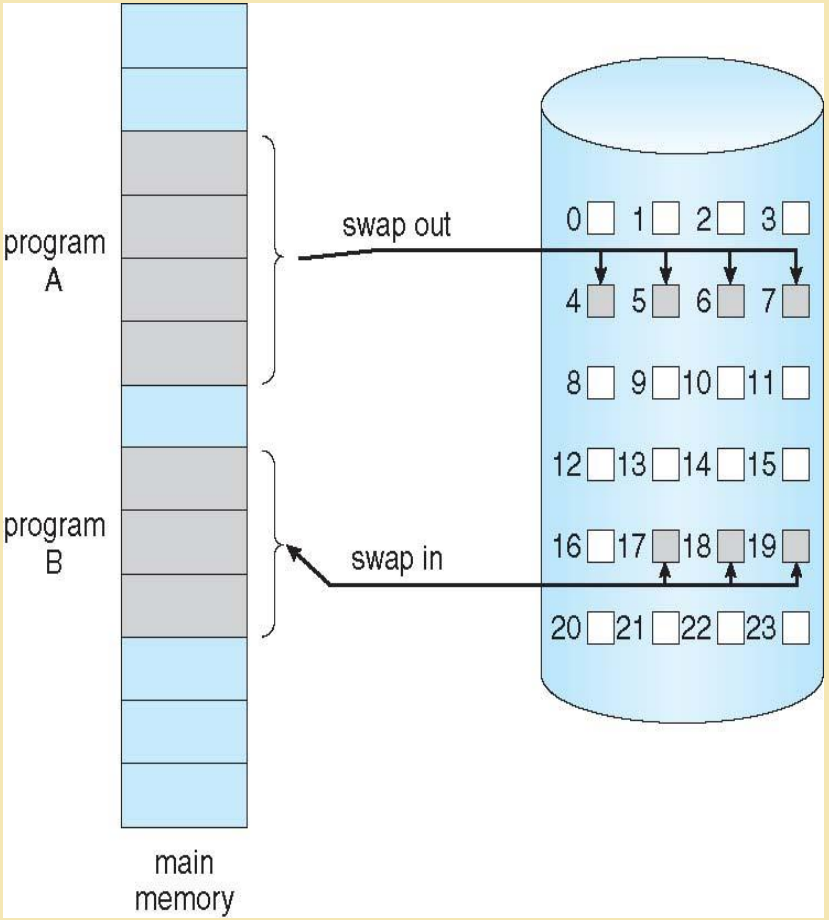


Thư viện chia sẻ sử dụng bộ nhớ ảo



Phân trang theo yêu cầu

- Có thể đưa toàn bộ tiến trình vào bộ nhớ tại thời điểm nạp
- Hoặc chỉ đưa một trang vào bộ nhớ khi nó cần thiết
  - Ít I/O cần thiết, không I/O không cần thiết
  - Cần ít bộ nhớ hơn
  - Phản hồi nhanh hơn
  - Nhiều người dùng hơn
- Tương tự như hệ thống phân trang có hoán đổi (sơ đồ bên phải)
- Page cần thiết ⇒ tham chiếu đến nó
  - tham chiếu không hợp lệ ⇒ huỷ bỏ
  - không có trong bộ nhớ ⇒ đưa vào bộ nhớ
- **Lazy swapper** - không bao giờ hoán đổi trang vào bộ nhớ trừ khi cần trang
  - Swapper xử lý các trang được gọi là **Pager**

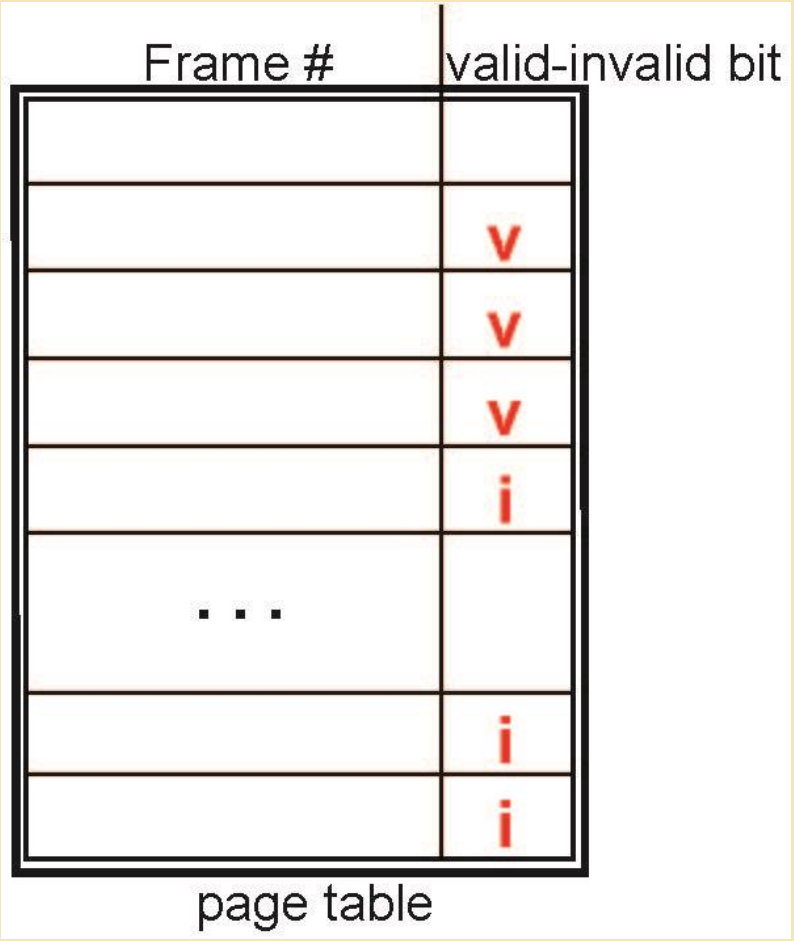


Phân trang theo yêu cầu

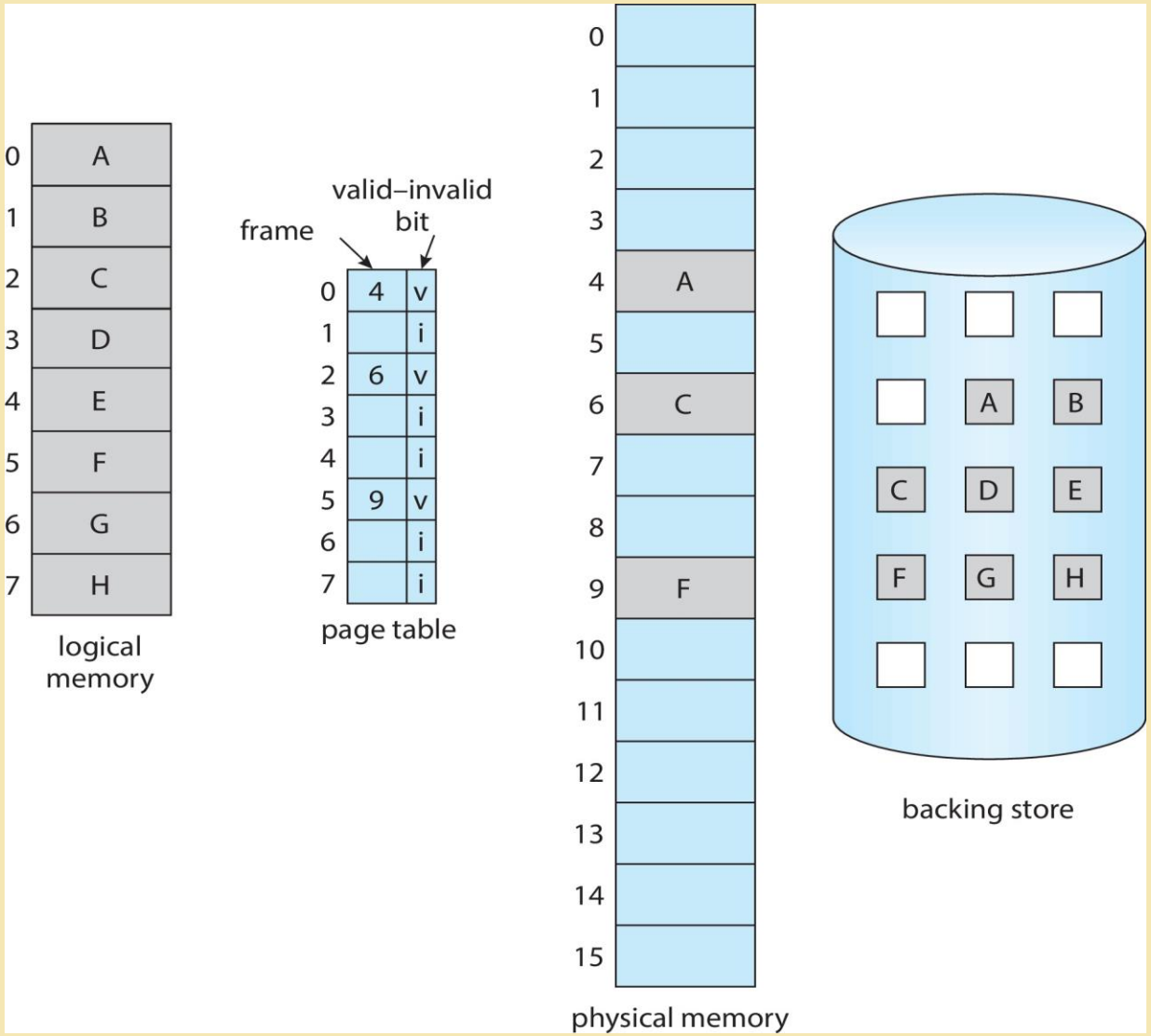
- Với tính năng hoán đổi, pager sẽ tính toán trang nào sẽ được sử dụng trước khi hoán đổi lại
- Pager chỉ đưa những trang đó vào bộ nhớ
- Làm thế nào để xác định tập hợp các trang thỏa mãn?
  - Cần chức năng mới của MMU để triển khai phân trang theo yêu cầu
- Nếu các trang cần thiết đã tồn tại trong bộ nhớ
  - Không có sự khác biệt so với không phân trang theo yêu cầu
- Nếu trang cần thiết và không tồn tại trong bộ nhớ
  - Cần dò tìm và nạp trang vào bộ nhớ từ bộ nhớ ngoài
    - Không thay đổi vị trí của chương trình
    - Không cần lập trình viên thay đổi mã

Bit hợp lệ-không hợp lệ (Valid-Invalid Bit)

- Với mỗi mục nhập bảng trang, một bit hợp lệ-không hợp lệ được liên kết
  (**v** ⇒ trong bộ nhớ , **i** ⇒ không trong bộ nhớ)
- Bit hợp lệ-không hợp lệ ban đầu được đặt là **i** trên tất cả các mục nhập
- Trong suốt quá trình MMU chuyển đổi địa chỉ, nếu bit hợp lệ-không hợp lệ trong mục nhập bảng trang là **i** ⇒ lỗi trang (page fault)

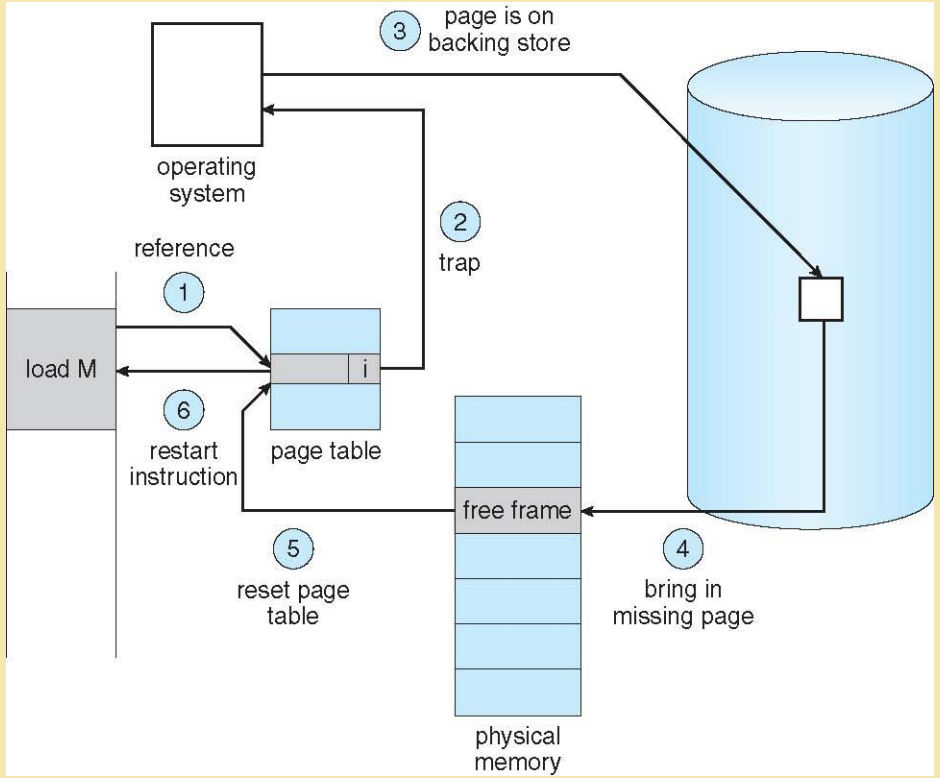


Bảng trang khi một số trang không nằm trong bộ nhớ



Các bước xử lý lỗi trang

1. Nếu có tham chiếu đến một trang, tham chiếu đầu tiên đến trang đó sẽ dẫn đến hệ điều hành
   - Page fault
2. Hệ điều hành tìm trong bảng khác để quyết định:
   - Tham chiếu không hợp lệ ⇒ hủy bỏ
   - Chỉ không có trong bộ nhớ
3. Tìm khung trống
4. Chuyển trang thành khung thông qua hoạt động lập lịch đĩa
5. Thiết lập lại bảng để cho biết trang đã có trong bộ nhớ
   Đặt bit xác thực = v
6. Khởi động lại lệnh đã gây ra lỗi trang



Danh sách khung trống

- Khi xảy ra lỗi trang, hệ điều hành phải đưa trang mong muốn từ bộ nhớ phụ vào bộ nhớ chính.
- Hầu hết các hệ điều hành duy trì một danh sách khung trống - một nhóm các khung trống để đáp ứng các yêu cầu như vậy.

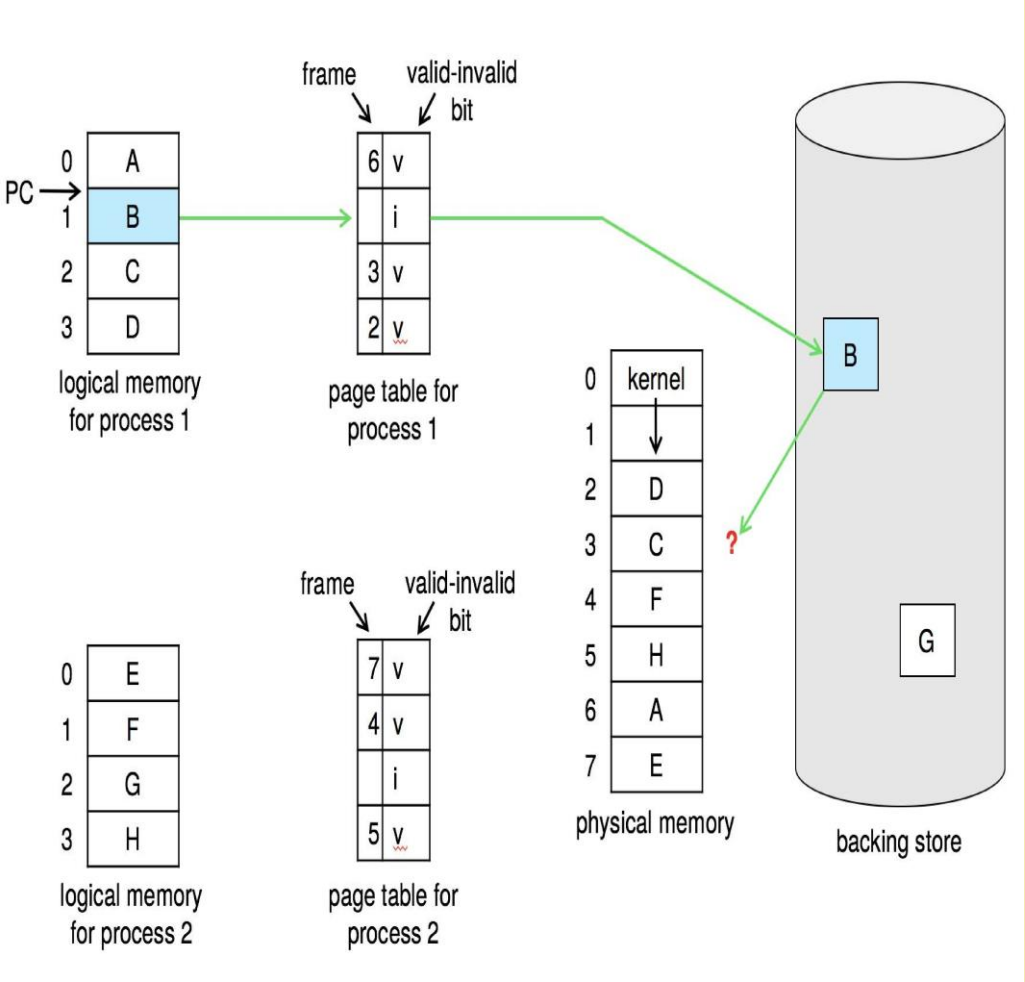
- Hệ điều hành thường phân bổ các khung trống bằng cách sử dụng một kỹ thuật được gọi là ***zero-fill-on-demand*** - nội dung của các khung được xóa sạch trước khi được cấp phát.
- Khi một hệ thống khởi động, tất cả bộ nhớ khả dụng sẽ được đưa vào danh sách khung trống.

Điều gì sẽ xảy ra nếu không có khung trống?

- Các trang tiến trình đã sử dụng hết
- Nhu cầu từ kernel, bộ đệm I/O, v.v.
- Bao nhiêu để cấp phát cho mỗi yêu cầu?
- Thay thế trang - tìm một số trang trong bộ nhớ, nhưng không thực sự được sử dụng, thay nó ra.
  - Thuật toán - kết thúc? hoán đổi? thay trang?
  - Hiệu suất - muốn một thuật toán dẫn đến số lỗi trang tối thiểu
- Cùng một trang có thể được đưa vào bộ nhớ nhiều lần

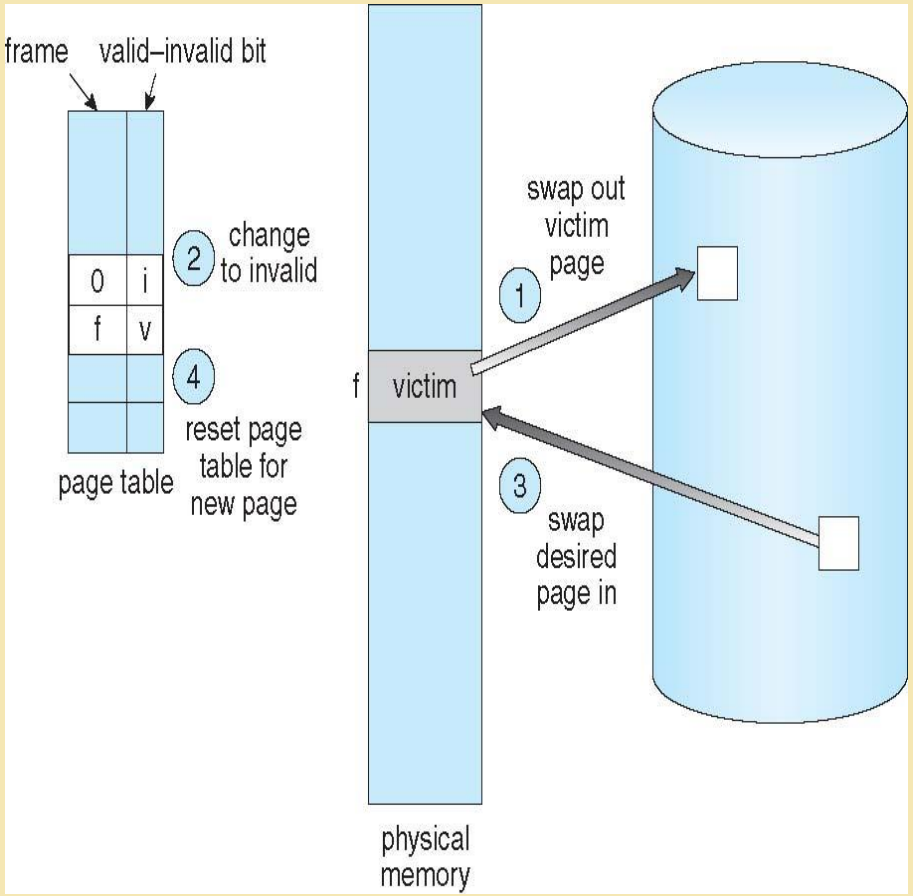
Thay thế trang

- Ngăn chặn cấp phát quá mức bộ nhớ bằng cách cập nhật dịch vụ lỗi trang, bao gồm thay thế trang
- Sử dụng bit sửa đổi (dirty) để giảm chi phí chuyển trang - chỉ các trang đã sửa đổi mới được ghi vào đĩa
- Việc thay thế trang hoàn thành việc phân tách giữa bộ nhớ logic và bộ nhớ vật lý - bộ nhớ ảo lớn có thể được cung cấp trên bộ nhớ vật lý nhỏ hơn



Thay thế trang

1. Tìm vị trí của trang mong muốn trên đĩa
2. Tìm khung trống:
   - Nếu có khung trống, thì sử dụng nó
   - Nếu không có khung trống, sử dụng thuật toán thay thế trang để chọn 1 khung nạn nhân (victim frame)
   - Ghi khung nạn nhân vào đĩa
3. Đưa trang mong muốn vào khung trống (mới); cập nhật bảng trang và khung
4. Tiếp tục quá trình bằng cách khởi động lại lệnh đã gây ra bẫy lỗi



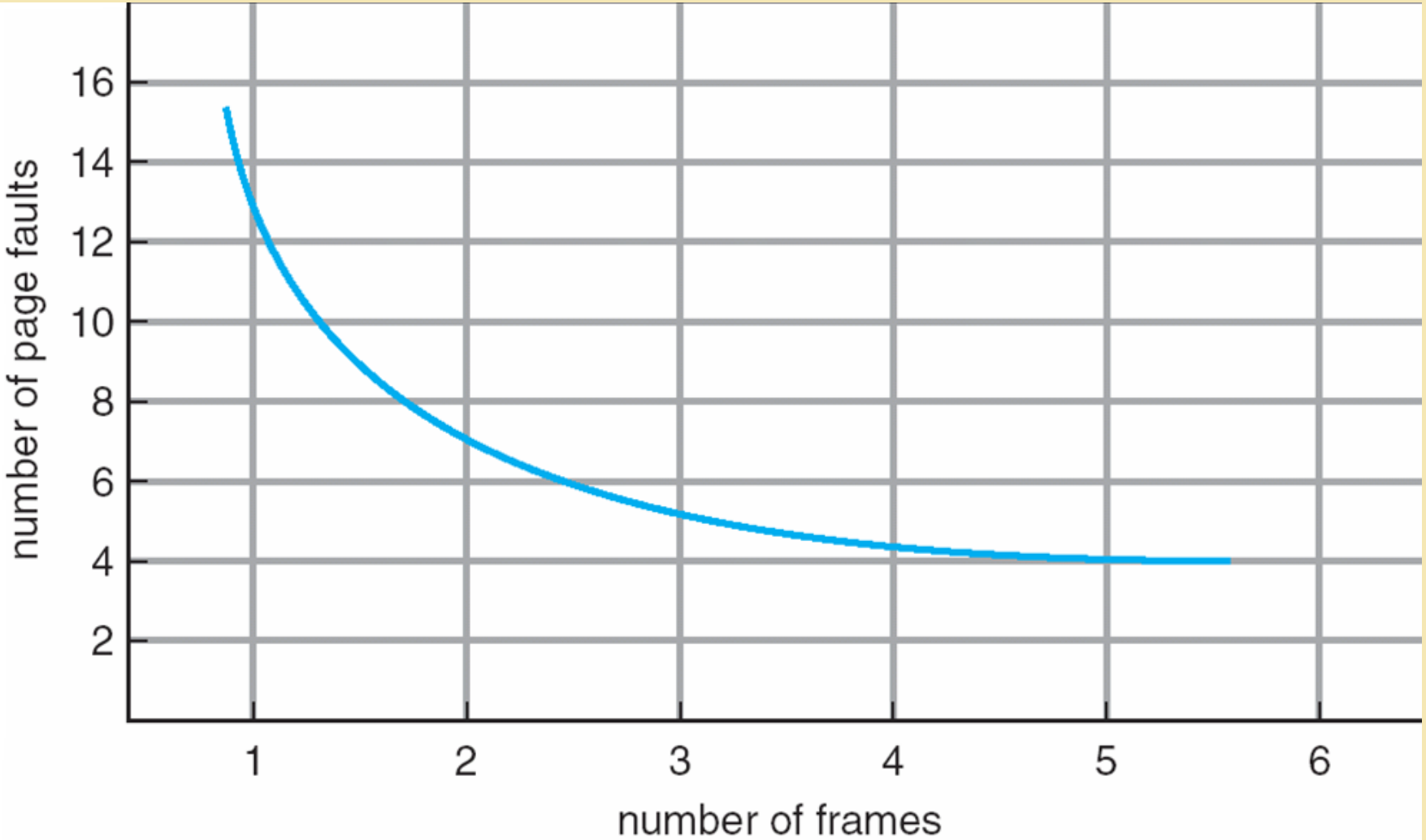
Các thuật toán thay thế trang và khung

- Xác định thuật toán cấp phát khung (**Frame-allocation algorithm**)
  - Bao nhiêu khung cho mỗi tiến trình
  - Những khung nào để thay thế
- Thuật toán thay thế trang (**Page-replacement algorithm**)
  - Muốn tỷ lệ lỗi trang thấp nhất trên cả lần truy cập đầu tiên và truy cập lại
- Đánh giá thuật toán bằng cách chạy nó trên một chuỗi tham chiếu bộ nhớ cụ thể (gọi tắt là chuỗi tham chiếu, reference string) và tính toán số lỗi trang trên chuỗi đó
  - Chuỗi chỉ là số trang, không phải địa chỉ đầy đủ
  - Việc truy cập nhiều lần vào cùng một trang không gây ra lỗi trang
  - Kết quả phụ thuộc vào số lượng khung trang có sẵn
- Ví dụ: hãy xem xét chuỗi địa chỉ sau: 123,215,600,1234,76,96

Nếu kích thước trang là 100, thì chuỗi tham chiếu là

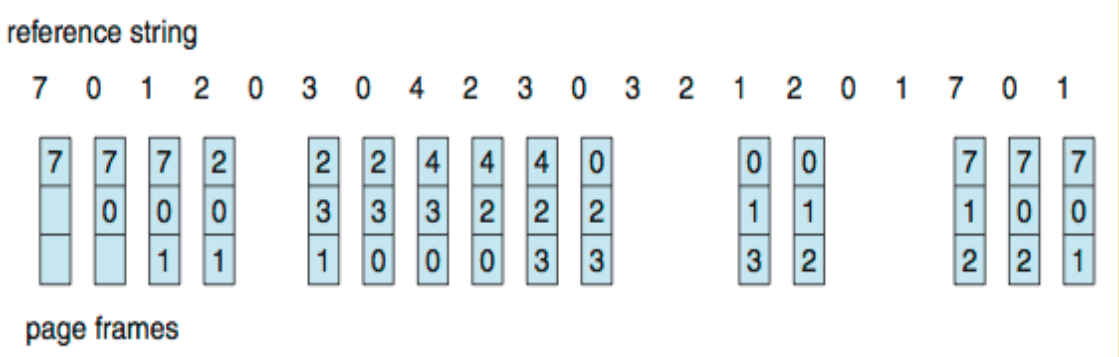
1,2,6,12,0,0

Đồ thị thể hiện số lỗi trang vs số khung



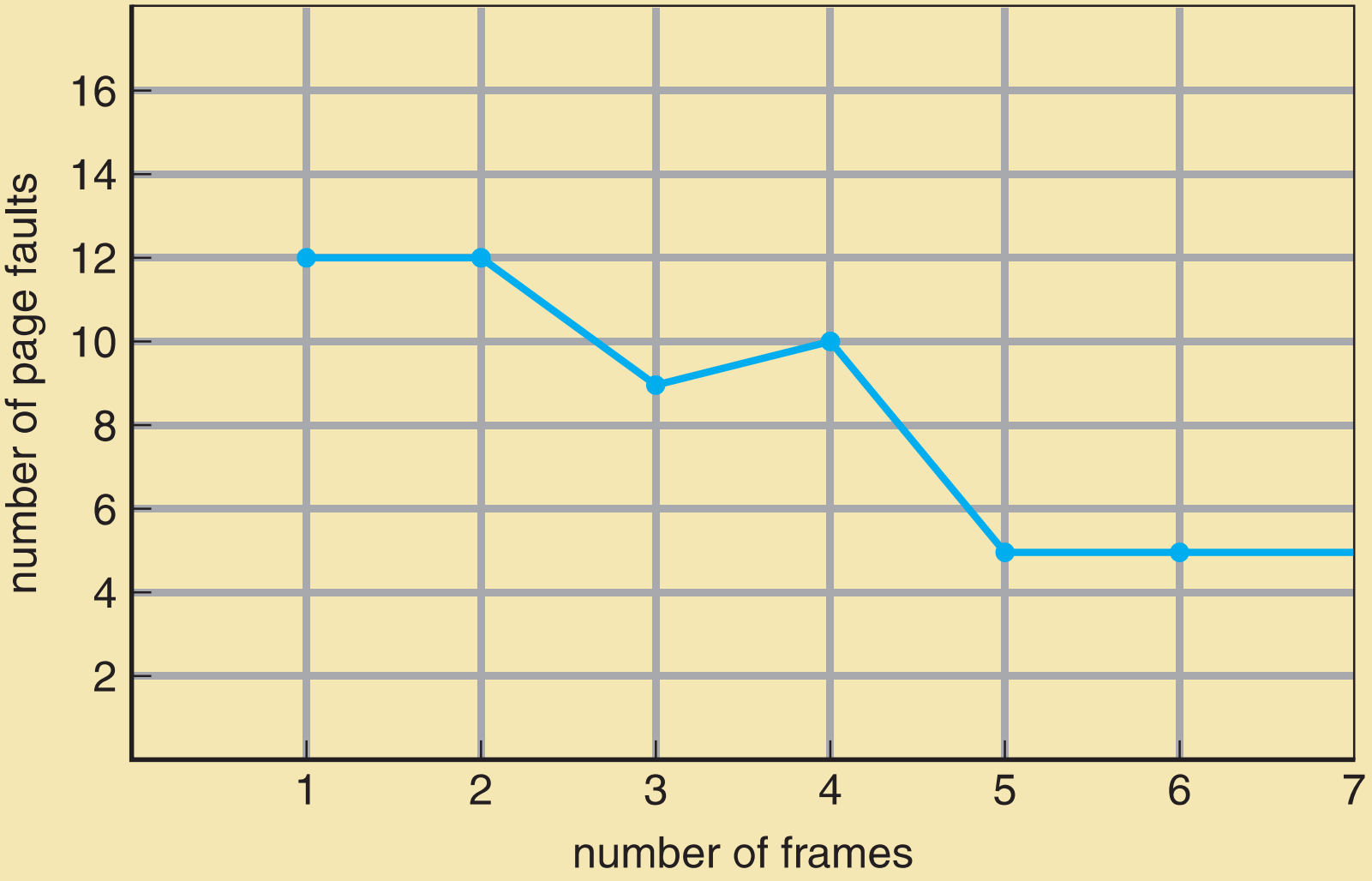
Thuật toán First-In-First-Out (FIFO)

- ***Trang cũ nhất trong bộ nhớ chính là trang sẽ được chọn để thay thế.***
- Xét chuỗi tham chiếu: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages trong bộ nhớ tại 1 thời điểm/ tiến trình)



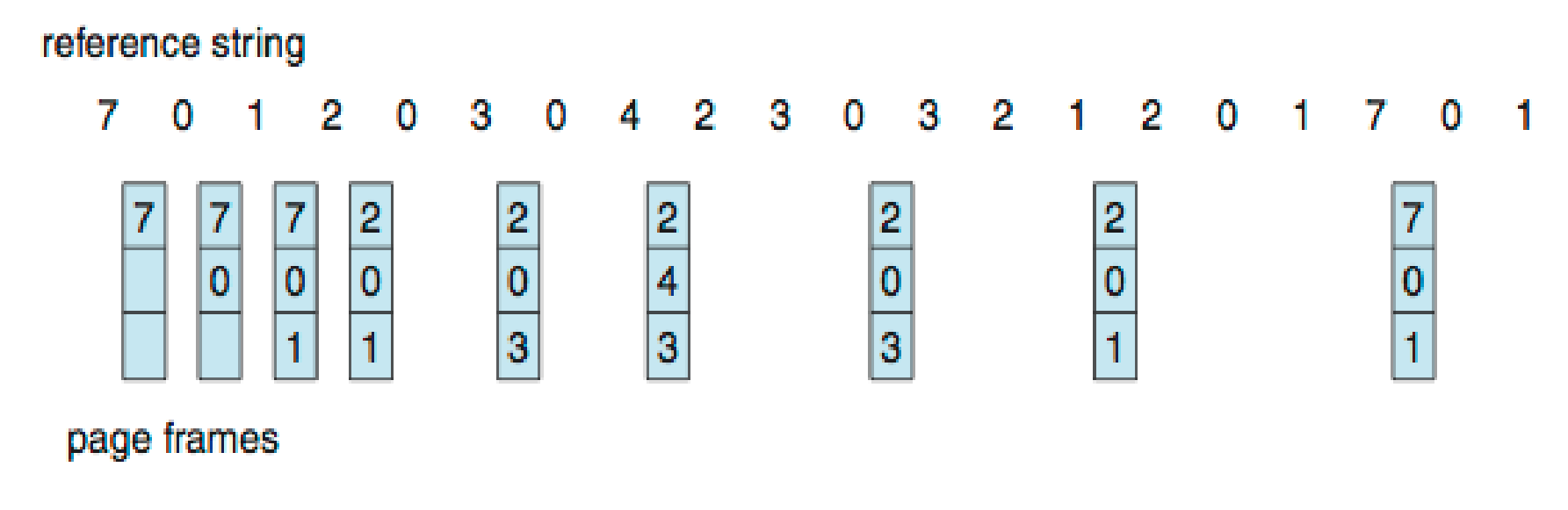
- Có thể thay đổi chuỗi tham chiếu: xem xét 1,2,3,4,1,2,5,1,2,3,4,5
  - Thêm nhiều khung có thể gây ra nhiều lỗi trang hơn!
    - **Nghịch lý Belady** (Thông thường, số khung tăng thì số lỗi trang giảm. Tuy nhiên, với FIFO số khung tăng thì số lỗi trang tăng theo)
- Làm thế nào để theo dõi trang cũ nhất?
  - Chỉ cần sử dụng hàng đợi FIFO

FIFO Minh họa nghịch lý Belady



Thuật toán OPT (Optimal Page Replacement)

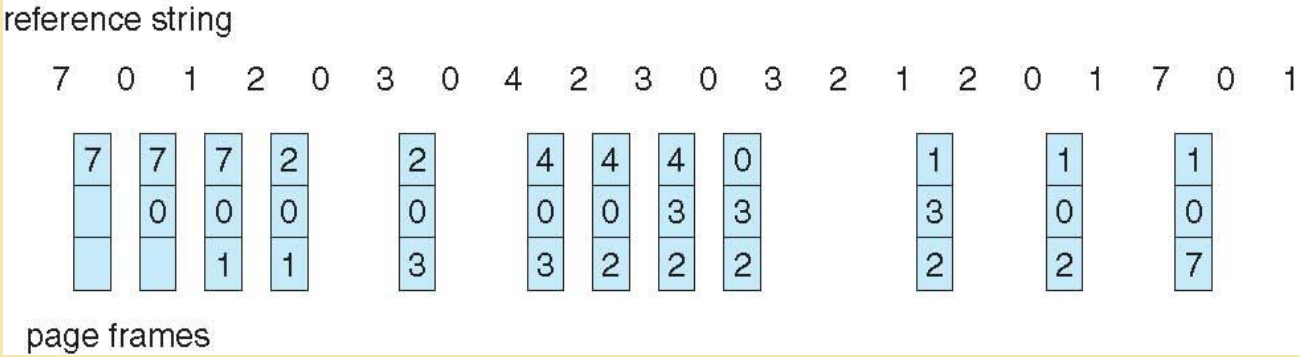
- ***Thay thế trang <u>sẽ</u> không được sử dụng trong thời gian dài nhất***



- Làm thế nào biết được điều này?
  - Không thể đọc tương lai
- Được sử dụng để đo lường một thuật toán hoạt động tốt như thế nào

Thuật toán LRU (Least Recently Used)

- Sử dụng trang trong quá khứ thay vì tương lai
- ***Thay thế trang không được sử dụng trong khoảng thời gian nhiều nhất***
- Liên kết thời gian sử dụng gần đây nhất với mỗi trang



- 12 lỗi trang – tốt hơn FIFO nhưng xấu hơn OPT
- Là thuật toán tốt và được sử dụng thường xuyên

Thắc mắc & Giải đáp